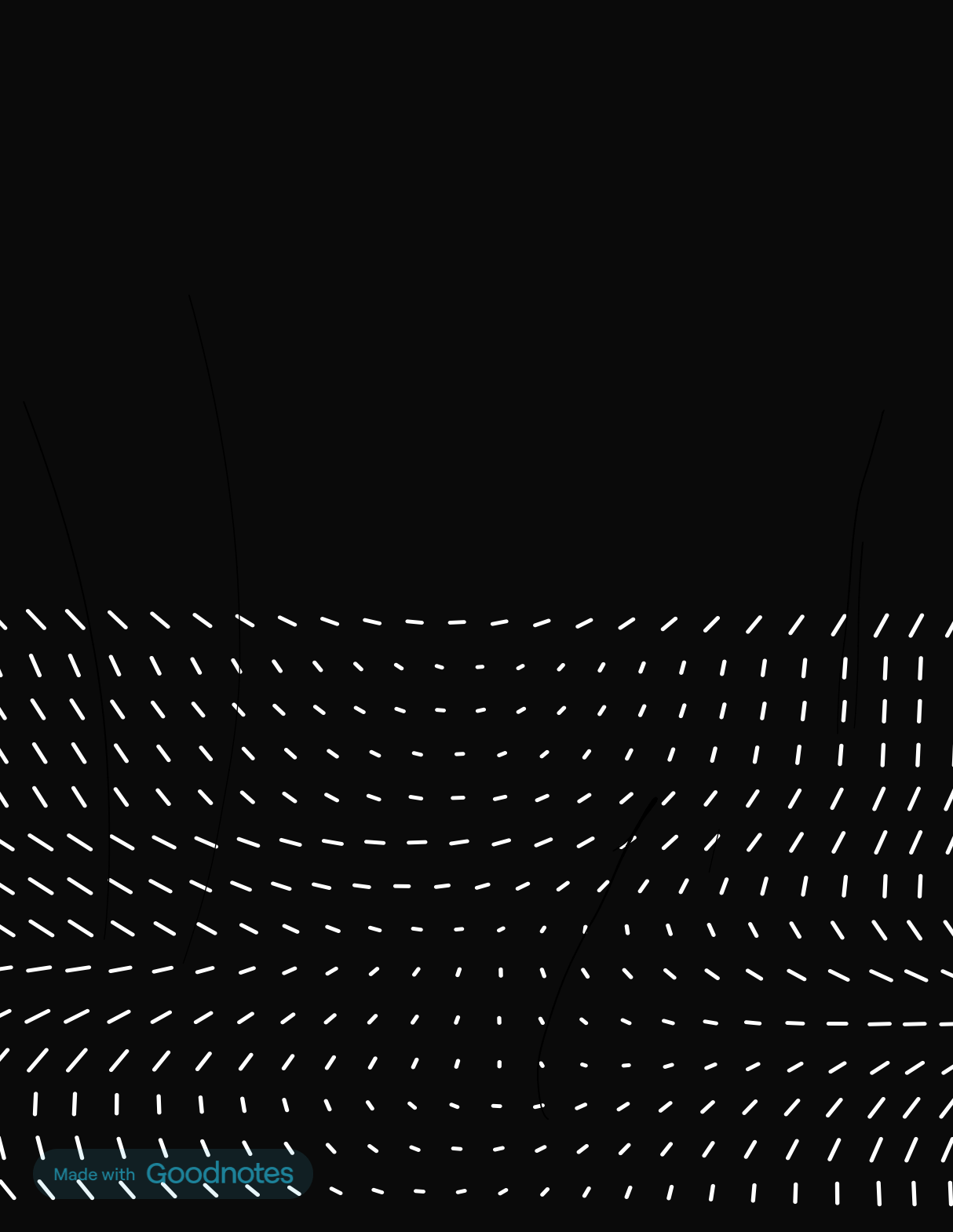

Made with Goodnotes

Computer Architecture → high level, What the system is.

Computer Organization ⇒ Low level, How is the system built.

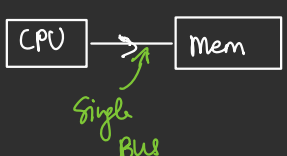
Hennessy Pattinson ? 5/6th Ed.

Computer Architectures

Von-Neumann Architecture

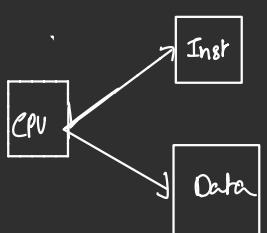
Same memory for instruction & data.

Disadv: Single bus for instruction & data
⇒ traffic in data
∴ slower access



Howard Architure

– 2 different memory spaces for instruction & data



- Used nowadays.

Levels of Memory

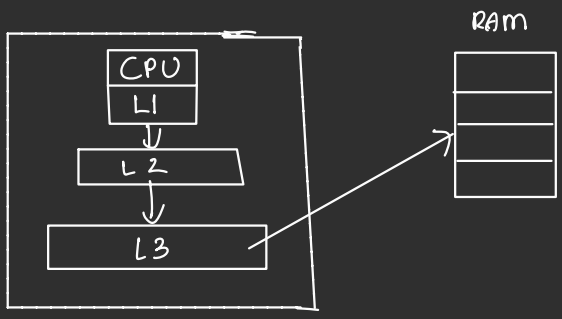
Register
↓
Cache
↓
Primary
↓
Secondary
↓
Hard disk.

Made with Goodnotes

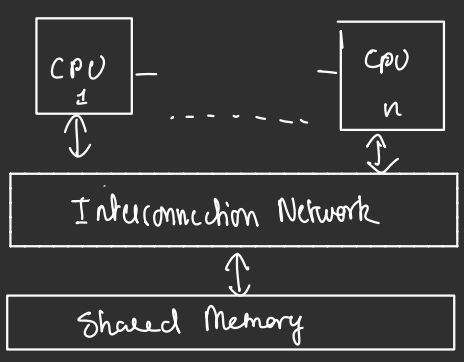
Levels of Cache

L1 Cache → present in processor.

L3 Divided into 2 segments, one for instructions, the other for Data.



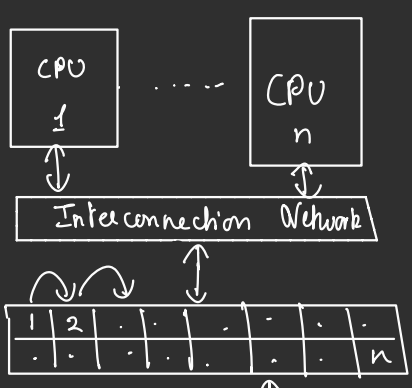
- Adv: Better average access time.
- Disadv: If it is a miss in L1, L2 & L3, then we must search RAM. ⇒ Latency ↑. Soln: Ditching L3.



Shared Memory Architecture

Uniform memory architecture/access

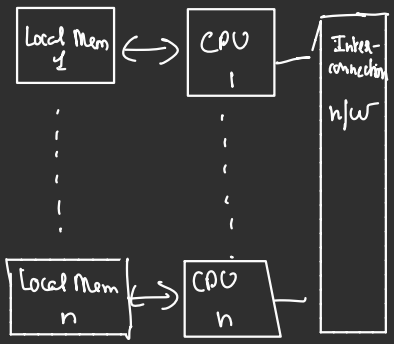
⇒ Access time for each CPU is almost equal



Shared Memory Divided into equal parts.

Distributed Shared memory Architecture

- Non Uniform memory access



① Distributed Memory Architecture. → NUMA

Tightly Coupled Systems:

1) UMA
2) NUMA
3) Cache Only Memory Access

Types of ISA

- RISC
  - Simpler
  - Less cycles ⇒ Less complexity ⇒ Less cost.
  - Uniform instruction size.

- CISC
  - Multiple operations in a single instruction.
  - No. of cycles / instructions ⇒ higher complexity.
  - Non uniform instruction size.

Made with Goodnotes

PARALLEL COMPUTERS

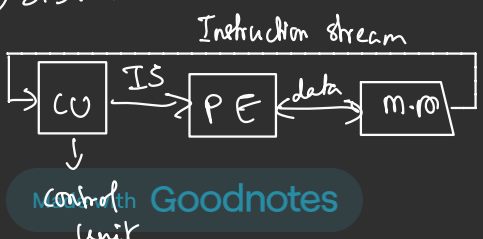
1) Processing element
2) Arrangement of memory module
3) Communication b/w memory module & processing element.
4) " " processing elements.
5) Nature of Processing elements.
6) Applications
7) Internal Circuit → Simple / Complex

Types:

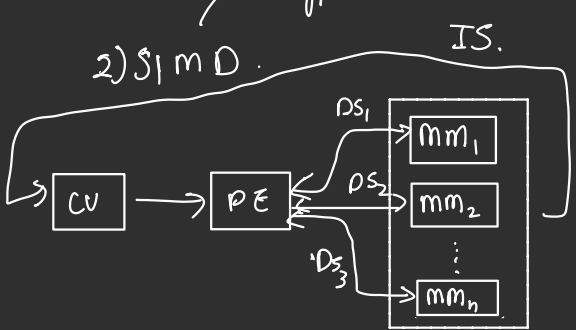
1) Flynn's Classification
2) Classification based on characteristics of memory modules.
3) Classification based on interconnection network.
4) Classification based on processing elements.
5) Special processor.

1) Flynn's Classification

1) SISD. AKA Von Neumann Architecture

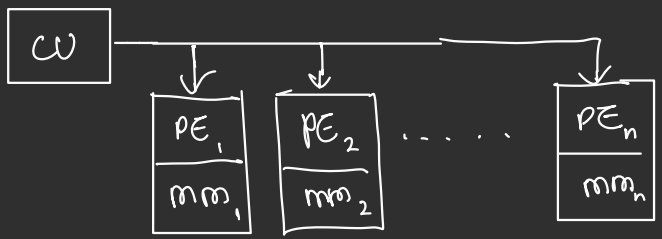


2) SIMD. → 2 Types: Vector & Array

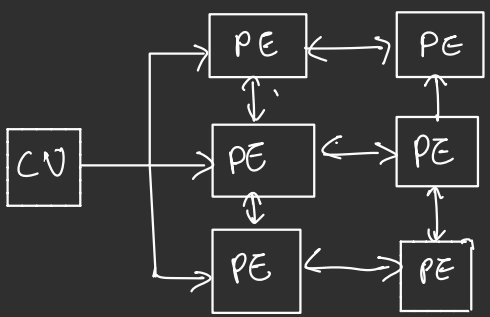


Goodnotes

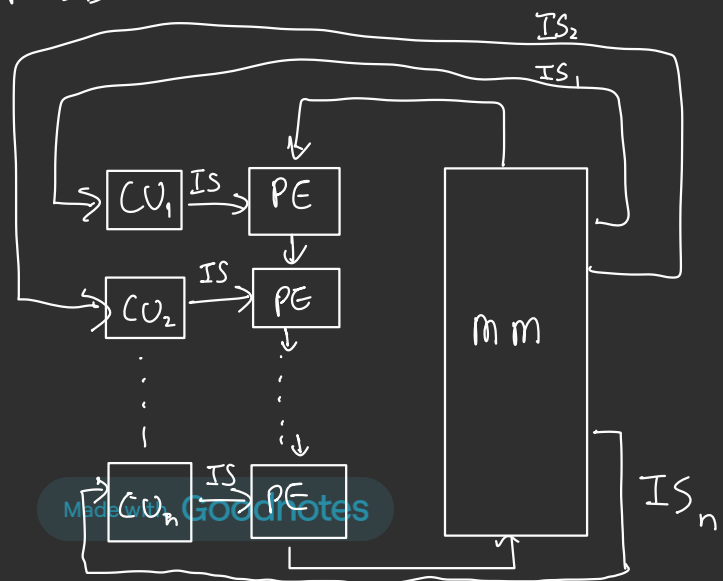
Vector SIMD



Array SIMD

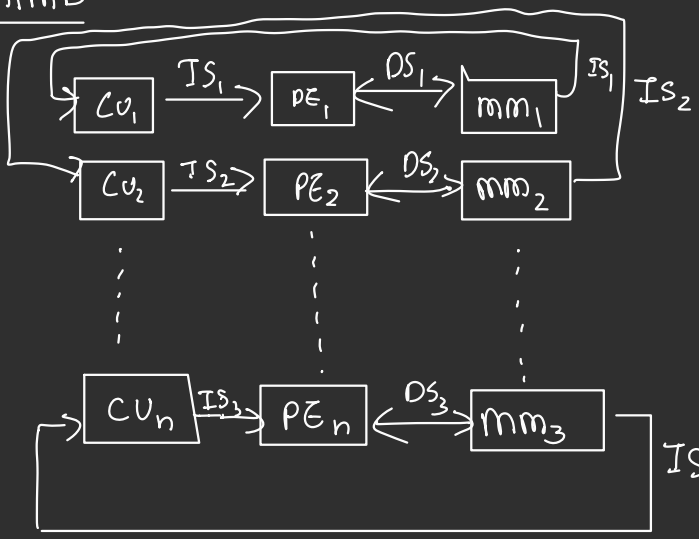


MISD



Made with Goodnotes

MIMD

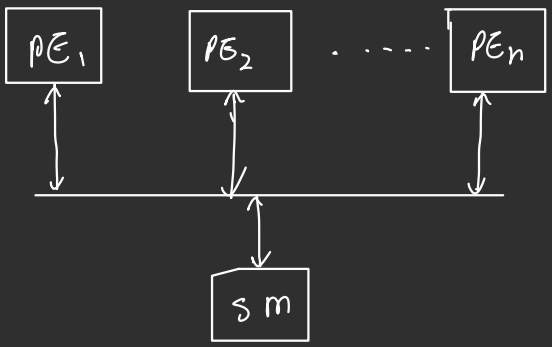


II) Based on Memory Modules.

1) Shared Memory Architecture. → UMA, NUMA, Cache only Memory Access.

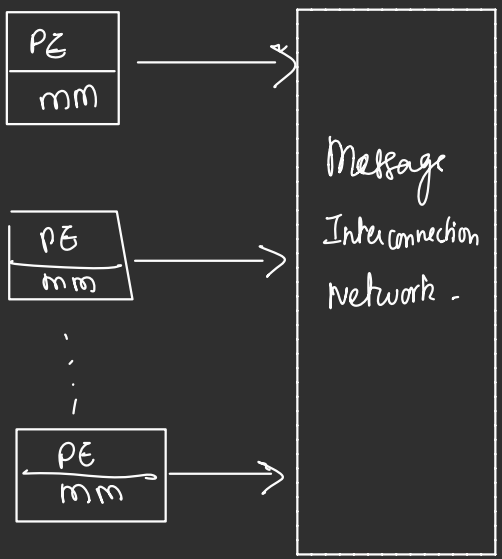
2) Distributed Memory Architecture

1) SMA



Made with Goodnotes

2) Distributed MA



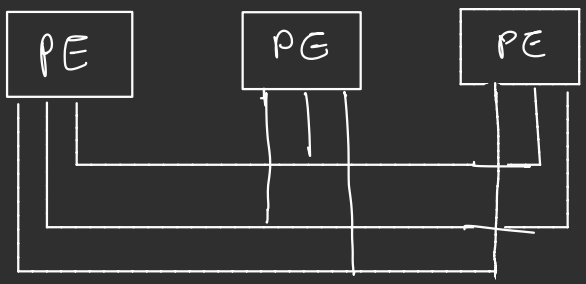
III) Based on Interconnection Network.

1) Linear

2) Bus.

Made with Goodnotes

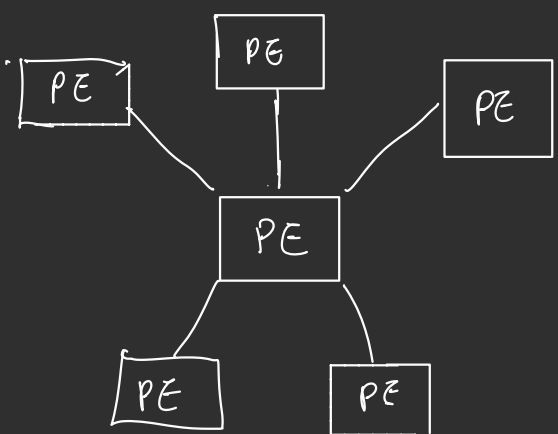
3) Multiple Bus



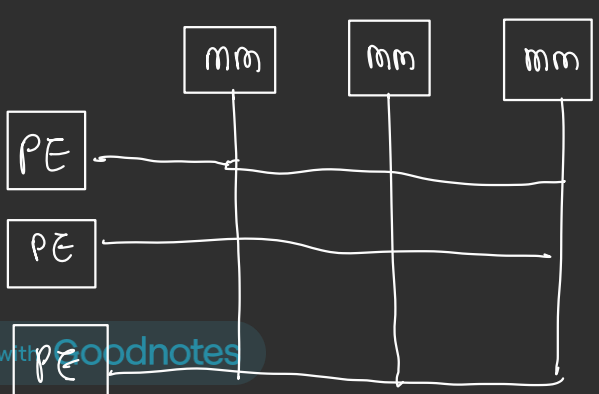
Disadv: Redundant Info

Adv: If one bus fails other bus still present.

4) Star Topology.

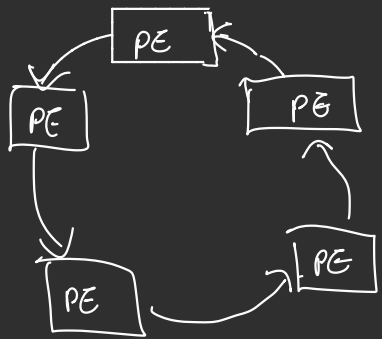


5) Crossbar Topology.

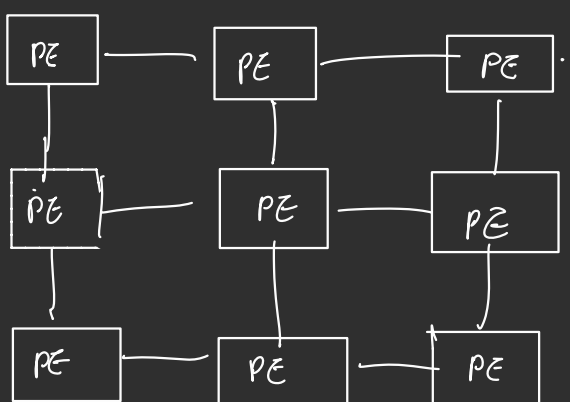


Made with Goodnotes

6) Ring Topology

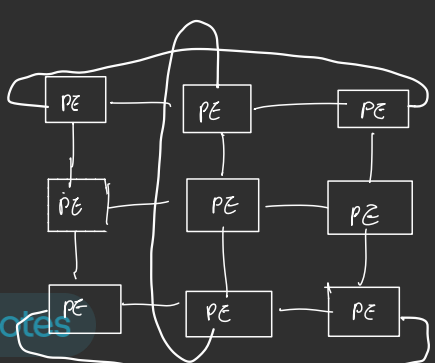


7) Mesh Topology



To pass info b/w 2 PEs, first move horizontally till you reach the column with the destination PE, then move upwards

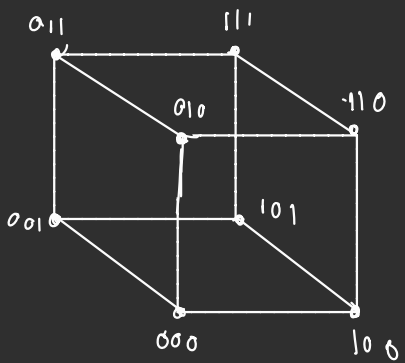
8) Torus



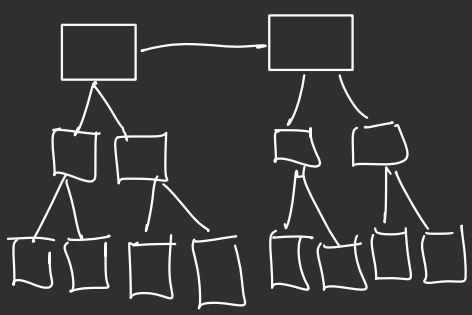
Reduces latency compared to mesh.

Made with Goodnotes

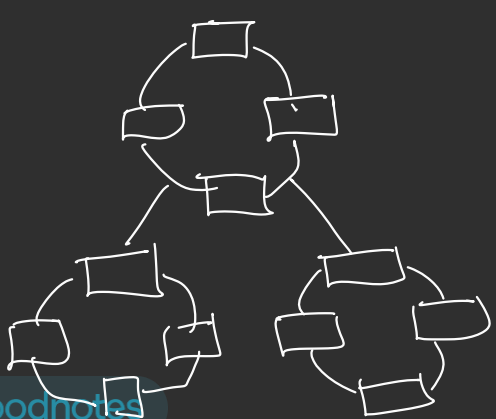
9) Hypercube



10) Flat/Fat Tree Architecture

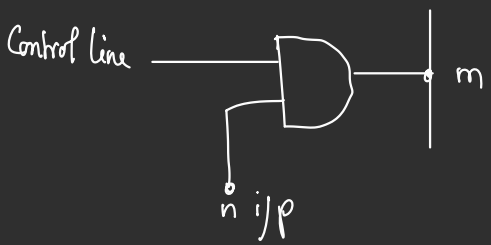


11) Hybrid Tree Architecture



Made with Goodnotes

II) Switch Topology



Direct Switch

Cross Switch

IV) Classifications Based On PEs

1) RISC VS CISC.

- RISC
  - Simpler
  - Less cycles ⇒ Less complexity ⇒ Less cost.
  - Uniform instruction size.
  - CPI = 1
  - Registers & Memory accessed using Load & Store.
  - Hardwired Control Unit ⇒ Fast, modify processor ckt whenever new instruction added

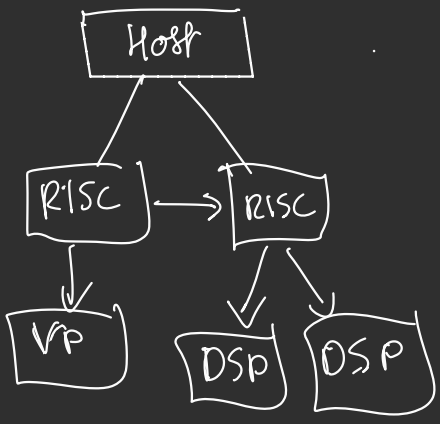
- CISC
  - Multiple operations in a single instruction.
  - No. of cycles / instructions ⇒ higher complexity.
  - Non uniform instruction size,

Made with Goodnotes

- Microprogrammed Control Unit → Control signals stored in control memory ⇒ slow as we need to access control memory whenever control signals needs to be accessed
- CPI > 1
- Difficult to program
- No pipeline / less pipeline
- Directly access memory ⇒ no LOAD | STORE.

2) DSP vs Vector Processor.
   ↳ For DSP applications [Image processing, sensor data etc.]

3) Heterogeneous vs Homogeneous Processor
   ↳ Used in embedded System.



Pipeline ⇒ 5 stages

Superscalar Pipeline ⇒ 20 stages.

Super pipeline → used in core i9 → P cores [Performance] → 14 stages
                                 → E cores [Efficiency] → 10 stages.

Made with Goodnotes

E cores used for background apps.

P cores → singlethreaded, E cores → multithreaded.

Pentium Pro → 10 stages.

Superscalar Pipeline → 3 instructions fetched at a time.

⇒ Instruction Level Parallelism ↑.

Super pipeline → Pipeline stages are divided into smaller stages

⇒ allows pipelining of instructions whose stages take different clock cycles

SPECIAL PROCESSORS

1) Pipeline
2) Multiple pipeline
3) Multiple SIMD.
4) Dataflow
5) Systolic & Wavefront Arrays.
6) SPMD.

I) Pipeline

↗ Identify the type of instruction to be executed [arithmetic, control, memory]

Pipeline → 5 stages [Fetch, Decode, Execute, memory, Write back]

[Either perform arithmetic ops, Find effective address for load/store / find branch address.

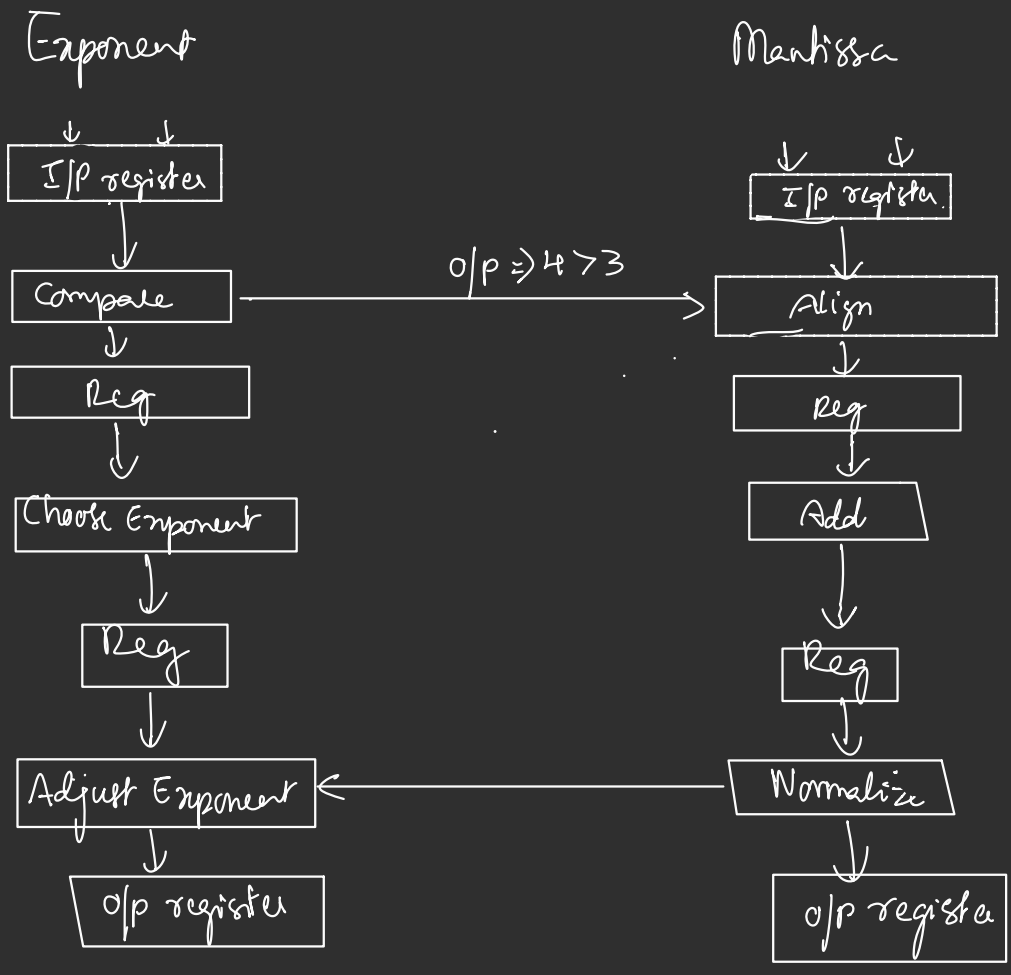
Made with Goodnotes

Addition of Floating Point Numbers

$a = 0.2345 \times 10^4$

$b = 0.7564 \times 10^3 = 0.07564 \times 10^4$

Compare exponent → align mantissa → add mantissa → Normalize



Made with Goodnotes

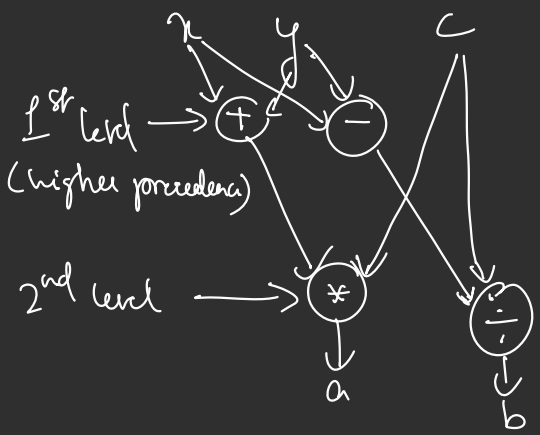
III) Multiple Pipeline ⇒ multiple integer pipelines ⇒ better ILP.

VI) Multiple SIMD ⇒ multiple SIMD connected together to form a MIMD ⇒ require multiple control units.

IV) Dataflow ⇒ Focus on given data.

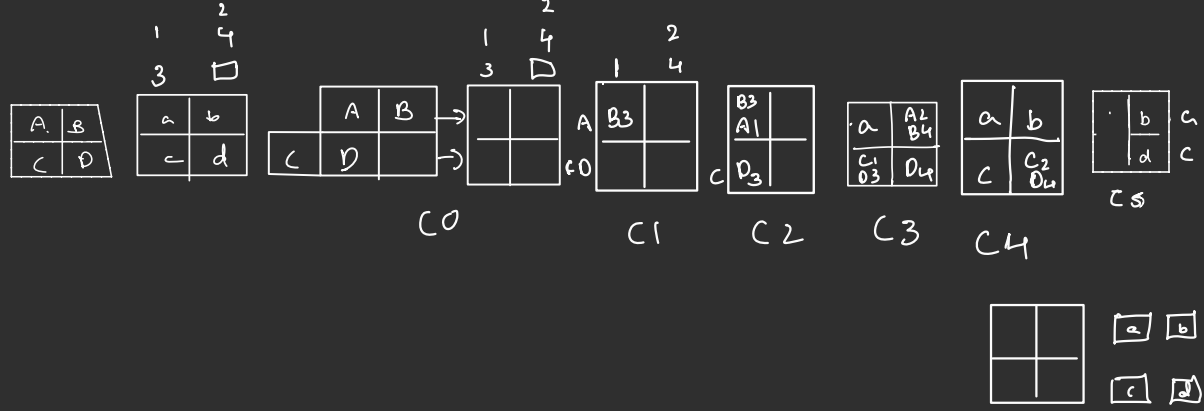
$a = (x+y) * c$

$b = (x-y) \div c$.



V) Single Program Multiple Data ⇒ Same program executed on different data [SPMD]. Requires MIMD processors.

Made with Goodnotes



Above → Systolic Arrays.

Development of ||el Computers

1) Moving from 4 bit & 8 bit each. to 16 bit, 32 bit & 64 bit each.

2) Instruction Level ||elism
   - dependent instructions
   - independent instructions
   - Out of order execution

   → Fine ILP → bit-level ILP
   → Data Level ||elism
   → Coarse ILP → Threaded level ILP

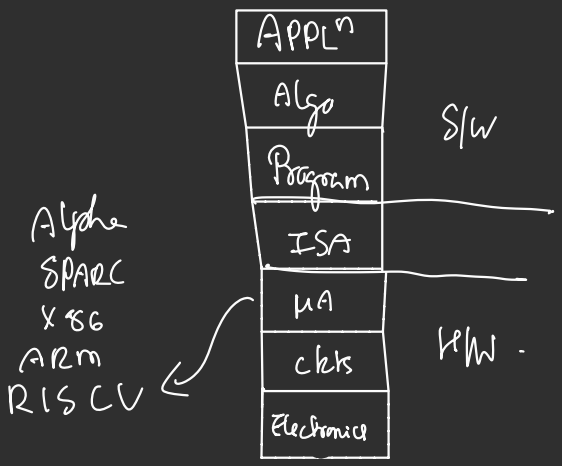
   Increase from 22MHz to 3.2 GHz.

3) No. of transistors double every 2 years ⇒ Very high power consumption & heat dissipation ⇒ Thermal issues.

4) Multicore ⇒ only certain cores are active, while others remain idle ⇒ lesser heat issues.

Made with Goodnotes

ISA's Interface b/w hardware & software.



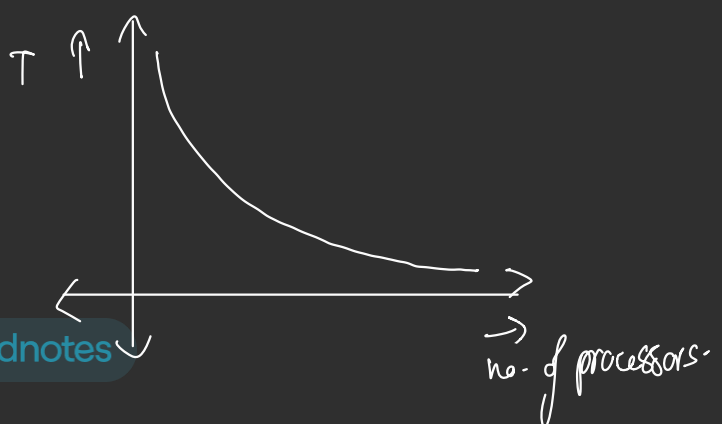
To execute n instructions

T_1 = 1 unit of time

no. of processors [unpipelined]

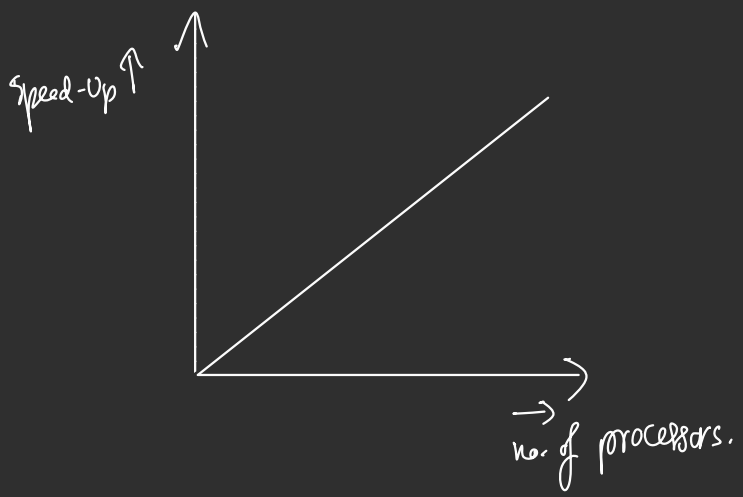
T_2 = 1/2 unit of time

T_3 = 1/3 unit of time.



Made with Goodnotes

$$\text{Speed-Up} = \frac{T_{old}}{T_{new}} = \frac{T_1}{T_2} = \frac{1}{\frac{1}{2}} = 2$$



However this cannot be replicated IRL as some instructions have to be executed in series [as they are dependent] while others are executed in n^{th}.

$$S + P = 1$$

$$S = 1 - P$$

$$T_1 = (1-P) + P$$

$$T_n = (1-P) + \frac{P}{n}$$

Made with Goodnotes

$$\text{Speed up} = \frac{1}{(1-p) + \frac{p}{n}}$$

(1−p): Serial, p/n: Parallel

2 types of instructions. Integer instructions (50%) & Floating point instructions (20%). Floating point instructions are executed using 3 cores. What is the speed up?

$$\text{Speed up} = \frac{1}{0.8 + \frac{0.2}{3}} = \frac{1}{\frac{24}{30} + \frac{2}{30}}$$

$$= \frac{30}{26} = \frac{15}{13}$$

Amdahl's Law → $\frac{T_{old}}{T_{new}} = \frac{1}{S+P} = \frac{1}{(1-P)+\frac{P}{n}}$

Components with high frequency → North Bridge [High speed].

Components with low frequency → South Bridge [Low speed]

Made with Goodnotes

Data Hazards. → a = b + c

c = a + d

Pipelining ⇒ ↑ Throughput.

i) Let a program have 40% of its code enhanced to run 23 times faster. What is its overall speed up? → 1.292

ii) Let a program have 40% of its code enhanced to yield a system speed up 43 times faster. What is the factor of improvement of the portion enhanced? (Not possible).

iii) Let a program have a portion of code enhanced to run 4 times faster to yield a system speed up 33 times faster. What is fraction enhanced?

i)

$$\frac{1}{0.6 + \frac{0.4}{23}} = \frac{1}{\frac{6}{10} + \frac{4}{230}} = \frac{1}{\frac{138}{230} + \frac{4}{230}} = \frac{230}{142} =$$

$$\frac{230}{142} = 1.62.$$

ii)

$$43 = \frac{1}{0.6 + \frac{0.4}{x}}$$

$$43 = \frac{x}{0.6x + 0.4}$$

Made with Goodnotes

$0.6x + 0.4 = \frac{x}{43}$

$0.4 = \frac{10x}{430} - \frac{6x}{10}$

iii) $33 = \frac{1}{(1-p) + \frac{p}{4}}$

132
-33

$33 - 33p + \frac{33}{4}p = 1$

$33 - \frac{99}{4}p = 1$

$32 = \frac{99}{4}p$

$\frac{4 \times 32}{99} = p$

$\frac{128}{99} = p$?

Made with Goodnotes

Dynamic Energy = Capacitance Load (no. of transistors) × Voltage^2.

$= \frac{1}{2} \times$ Capacitance Load $\times V^2$.

↳ Only consider 1/2 of the transitions?

↓

0→1→0 or 1→0→1

Power$_{Dynamic}$ = Energy$_{Dynamic}$ × Frequency of switching.

1) DVFS → Different Voltage Frequency Scaling → Some cores are supplied less voltage (less frequency) as they are executing low-priority tasks

2) Ideal Case:

not run → off

run → supply voltage.

3) Design for typical case

4) Overclocking → increasing frequency of a specific core so as to speed up task.

Made with Goodnotes

Cost of an IC =

$$\frac{\text{Cost of die} + \text{cost of testing die} + \text{cost of packaging \& final test}}{\text{final yield test}}$$

$$\text{Cost of die} = \frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{die yield}}$$

LS Imp.

$$\text{Dies per wafer} = \frac{\pi(\text{Wafer dia})^2}{\text{Die area}} - \frac{\pi d}{\sqrt{2 \times \text{Die area}}}$$

↳ Faulty dies

SLA ⇒ Service Level Agreement

① Service accomplishment ⇒ Good ① → ② ⇒ Failure

② Service Interruption ⇒ Failure ② → ① ⇒ Restoration.

Module Reliability ⇒ Measure of service accomplishment.

Mean Time To Failure [MTTF]

Service Interruption ⇒ Mean Time To Repair. [MTTR]

Mean Time B/W Failure = MTTF + MTTR

$$\text{Module Availability} = \frac{MTTF}{MTTF + MTTR}$$

Made with Goodnotes

Die yield = $\frac{\text{Wafer yield} \times 1}{(1 + \text{defects per unit area} \times \text{die area})^N}$ N = process complexity.

$\frac{1}{MTTF}$ = Failure Rate (λ)

Failure in Time [FIT] ⇒ for 1 billion hours.

↑ $\frac{10^9}{MTTF}$

$MTTR = \frac{\text{Down Time}}{\text{No. of Failures}}$

Q1). MTTF = 900 hours.
MTTR = 36 hours
MTBF = ?

If you require the system to have a 95% availability, how much downtime/1000 hours of operations can you tolerate?

Q2) A hard drive manufacturer tests 10 identical drives. They run continuously for 5000 hours. During the test, 2 hard drives fails at the 2500 & 4000 hour marks and are replaced. Calculate: MTTF, λ, FIT.

Made with Goodnotes

Q3) A web service is hosted on 2-identical systems in parallel. MTTF = 200h, MTTR = 20 hours, MTBF = ?.

Q4) 8 inch wafer, cost = 1000$, die area = 1 cm^2, defect density = 1 defect/cm^2.

Calculate dies/wafer, die yield, cost per die.

Q5) i) Wafer cost = 9000$, wafer dia = 30 cm, die area = 60 mm^2

defect density = 0.23 defects/cm^2.

Calculate dies/wafer, die yield.

ii) Using same wafer cost & defect density, calculate cost per good die for die area of 120 mm^2 & compare it cost of 60 mm^2.

iii) Suppose a new fabrication process is introduced that improves the defect density to 0.15 defects/cm^2 while keeping the die area to 60 mm^2. Calculate dies/wafer, die yield & cost per good die.

Made with Goodnotes

Q1) MTBF = 936 hours.

A = 0.909

Un. A = 1 - A = 0.091

Un. A for

PERFORMANCE METRICS

X is 'n' times faster than Y

$$\Rightarrow n = \frac{\text{Execution of Y}}{\text{Execution of X}}$$

$$\text{Performance} = \frac{1}{\text{Execution time}}$$

$$\Rightarrow n = \frac{\text{Performance of X}}{\text{Performance of Y}}$$

CPU Time = Clock cycles of a program x Clock Cycle Time.

$$= \frac{\text{Clock cycles of a program}}{\text{Clock Frequency.}}$$

Clock cycles of a program = Cycles per instructions x Instruction Count.

$$CPI = \frac{\text{no. of cycles}}{\text{Instruction Count.}}$$

Made with Goodnotes

CPU Time = IC × CPI × Clock cycle Time

$$= \frac{IC \times CPI}{\text{Clock Freq}}$$

Instruction Type	No. of Instruction	CPI
ALU	10	1
Load/Store	20	2
Control	30	3
Other	26	4

$$CPI_{avg} = \frac{1\times10 + 20\times2 + 30\times3 + 20\times4}{10+20+30+20}$$

$$\therefore CPI_{avg} = \frac{\sum_{i=1}^{n} IC_i \times CPI_i}{\sum_{i=1}^{n} IC_i}$$

MIPS → Million Instructions per second.

↳ $\boxed{\frac{n}{t \times 10^6}} = \frac{\cancel{I}}{\frac{\cancel{I} \times CPI}{Freq} \times 10^6} = \boxed{\frac{Freq}{CPI \times 10^6}}$.

Unit of CPU time = seconds/program.

Made with Goodnotes

CPU Time = IC × CPI × CCT

$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instructions}} \times \frac{\text{sec}}{\text{Clock Cycles}}$

= sec/program.

IC = ISA + Compiler Technology

CPI = ISA + Organization

Clock Cycle Time = Organization + Hardware.

Q1: Suppose that when program A is run, user CPU time = 3s. The elapsed (wall clock) time is 4s & system performance is 10 mflop/s. Assume that there are no other processes taking any significant amount of time & the computer is either doing calculations in the CPU or I/O but it cannot do both @ the same time. We now replace the processors with one that runs 6x faster. but doesn't affect the I/O speed. What will be the CPU time, wall clock time & mflop performance.

Q2: You are on the design team for a new processor. $f_{processor}$ = 200 MHz. The following table gives instruction frequencies for benchmark 3

Made with Goodnotes

Instruction Time	Frequency	Cycles
Load/Store	30%	6
ALU	50%	4
Others	20%	3

Processor executes one instruction @ a time.

Calculate:

i) CPI

ii) CPU execution time = 11s, MIPS?

iii) If Cycle Time ↑ by 20%, what is new clock freq?

iv) The compiler expert says if you double the no. of registers, then the compiler will generate code which requires half the no. of Load & Stores. What is the new CPI?

v) If we double the no. of registers, what will be the no. of CPU seconds?

Made with Goodnotes

1)
CPU time = 3s
I/O Time = 1s

$$\frac{1}{2} = \frac{x}{3} \quad x = 0.5s$$

new CPU time = 0.5s.
new WCT = 1.5s.
new Sys Perf = 60 mflops / 26.67 mflops

2)

i) $6 \times 0.3 + 4 \times 0.5 + 3 \times 0.2$
$= 1.8 + 2 + 0.6$
$= 4.4$ CPI

ii) $\frac{200 \times 10^6}{4.4 \times 10^6} = \frac{2000}{44} = 45.45$

iii) $\frac{1}{200 \times 10^6} = \frac{1}{2} \times 10^{-8}$
$= 5 \times 10^{-9}$
$= 6 \times 10^{-9}$
$= \frac{10^9}{6}$
$= 0.166 \times 10^9$
$= 166.66$ MHz.

Made with Goodnotes

iv) $\frac{0.15\times6+0.5\times4+0.2\times3}{0.85} = \frac{0.9+2+0.6}{0.85} = \frac{3.5}{0.85}$

$85\overline{)350}$ → 4.1, -340, 100

$\begin{array}{r} 85 \\ \times\ 4 \\ \hline 340 \end{array}$

$= 4.1$ CPI.

v) $\frac{\cancel{M}}{x} = \frac{0.4}{4.1}$

$\frac{4.1}{0.4} = \frac{10.25}{x}$

$10.25s$.

Made with Goodnotes

UNIT - 2

CHARACTERISTICS OF PIPELINE

1) Cannot decrease latency, however increases throughput.
2) Performance of pipeline dependent on slowest stage.
3) Speedup dependent on the number of pipeline stages.
4) Filling & draining of pipeline also affects performance.

Space - Time Diagram

	C_1	C_2	C_3	C_4	C_5
S_1	I_1	I_2	I_3	I_4	I_5
S_2		I_1	I_2	I_3	I_4
S_3			I_1	I_2	I_3
S_4				I_1	I_2
S_5					I_1

5 instructions ⇒ 9 clock cycles.

Speedup = $\frac{25}{9}$.

CPI = $\frac{9}{5} = 1.8$.

Eff^n = $\frac{2.5}{5} = 0.5$.

No. of stages : 4
Speedup : $\frac{25}{8}$

no. of stages = 6
Speedup : $\frac{25}{10}$.

Made with Goodnotes

Assumptions

1) Uniform Subcomputation [Latency of each is every stage]

Reality: Memory reference instructions take higher duration.

⇒ Reduce the memory addressing modes.

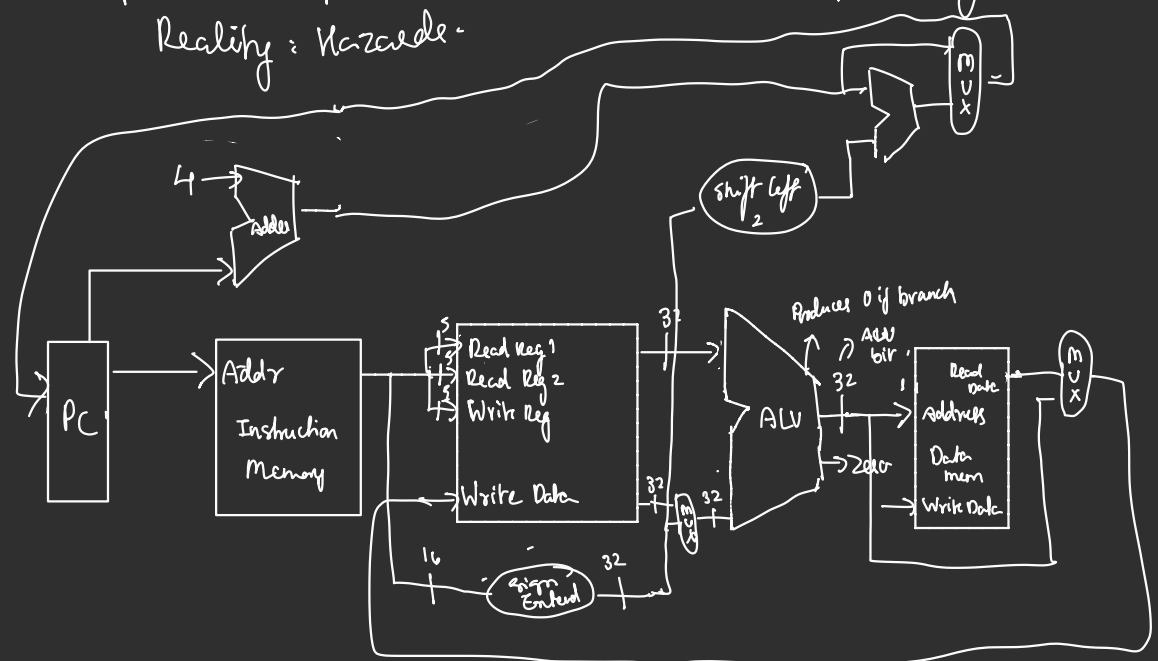
2) Identical Computation. [Each stage executes the same thing once & over again]

Reality: One clock cycle, ALU performs arithmetic, another clock cycle it is computing the memory reference address. ALU → skips MEM.

⇒ Keep Instruction Format Simple.

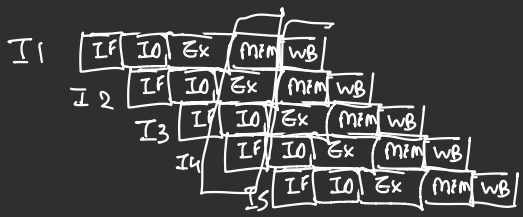
3) Independent Computation [Instructions are independent of each other]

Reality: Hazards.



IF → Get instruction address from PC, also PC = PC + 4 → Address of next address.

ID → decode the types of instruction, also find the registers to read & write from.



If Instruction Memory & Data memory not separate ⇒ Structural Hazard

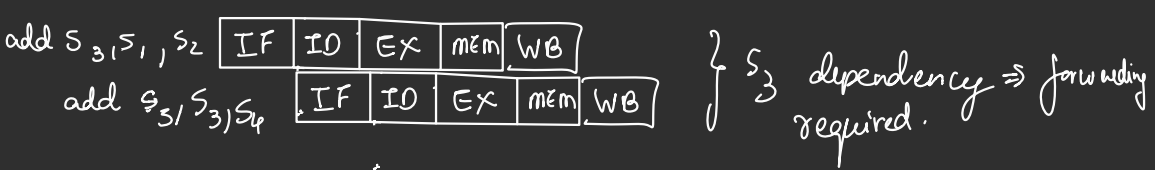
In the 5th clock cycle when I_1 is in WB, I_4 is in ID ⇒ Both I_1 & I_4 access register file ⇒ Data Hazard [Read after Write Hazard]

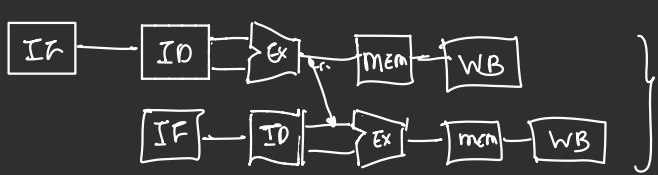
⇒ First half of clock cycle ⇒ read, 2nd Half of clock cycle ⇒ write.

ALU/Load → 5 Clock Cycles.

Store → 4 Clock Cycles.

Branch → 2 Clock Cycles [To identify whether instruction is for branching or not].

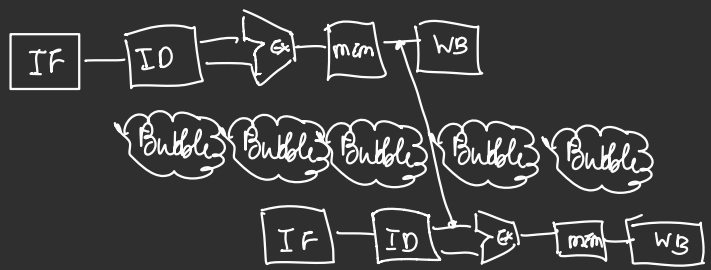




Made with Goodnotes

load S_1, offset

add S_3, S_1, S_2.



IF/ID pipeline register → 64 bit wide, stores 2 instruction addresses.

LOAD vs STORE

Writeback is idle during store while active during load.

FIGURE 5.1

To find out whether a 2 consecutive instructions will lead to a data hazard, a hardware unit is present b/w the Decode & Execute Stage which is connected to the Decode & Writeback hardware. For forwarding, pipeline register connected to forwarding unit.

Branch Hazards

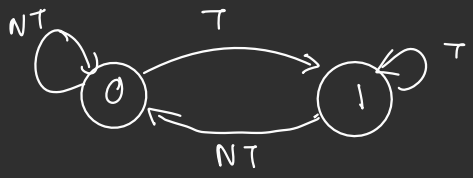
1) From target
2) From fall through

}?

Made with goodnotes

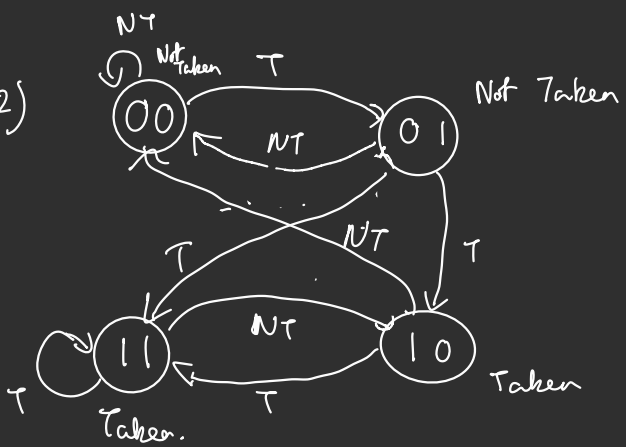
Branch Predictor

1) One Bit Predictor.



1 → 1 → 1 → 0 → 1 → 1

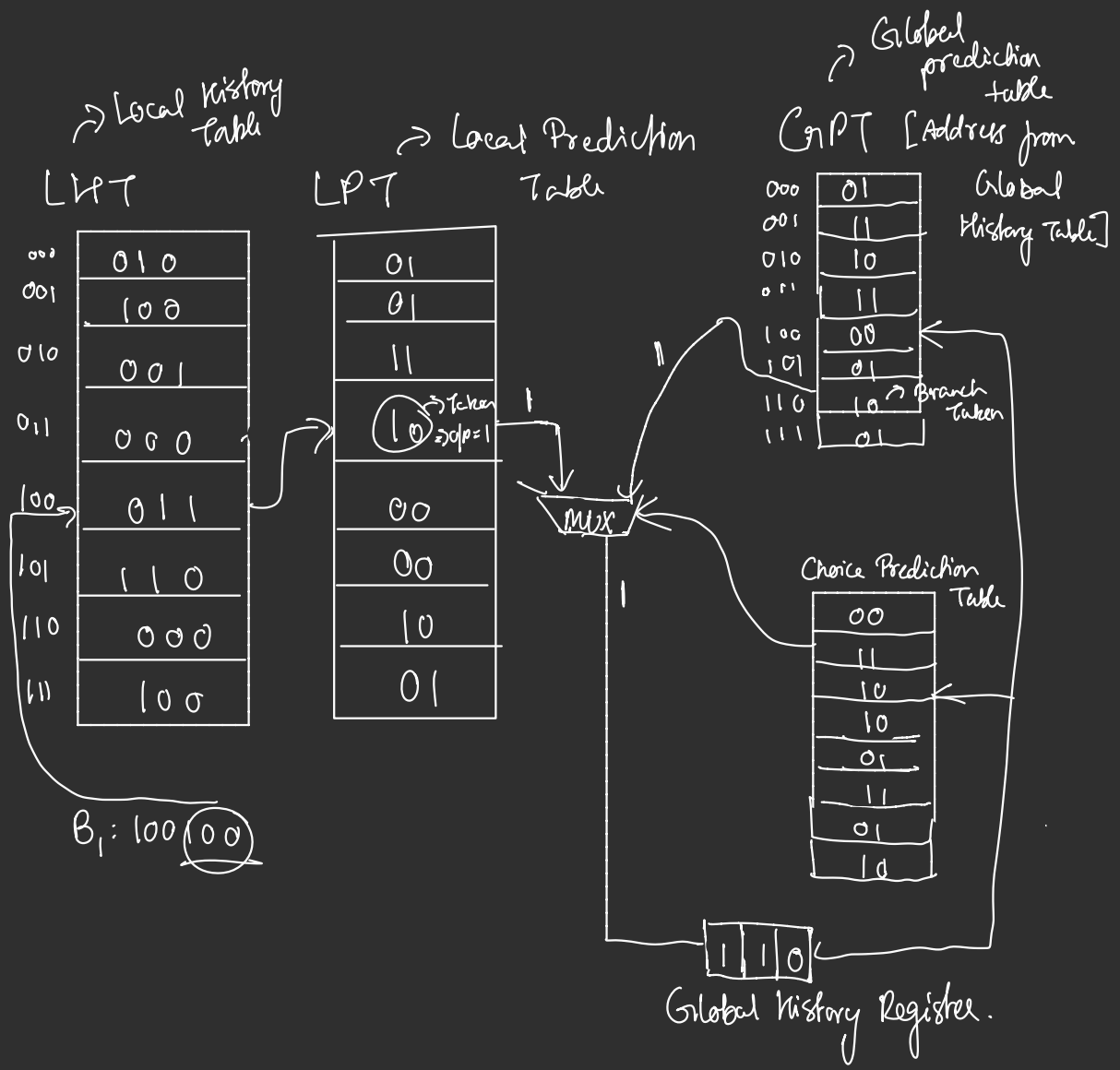
2)



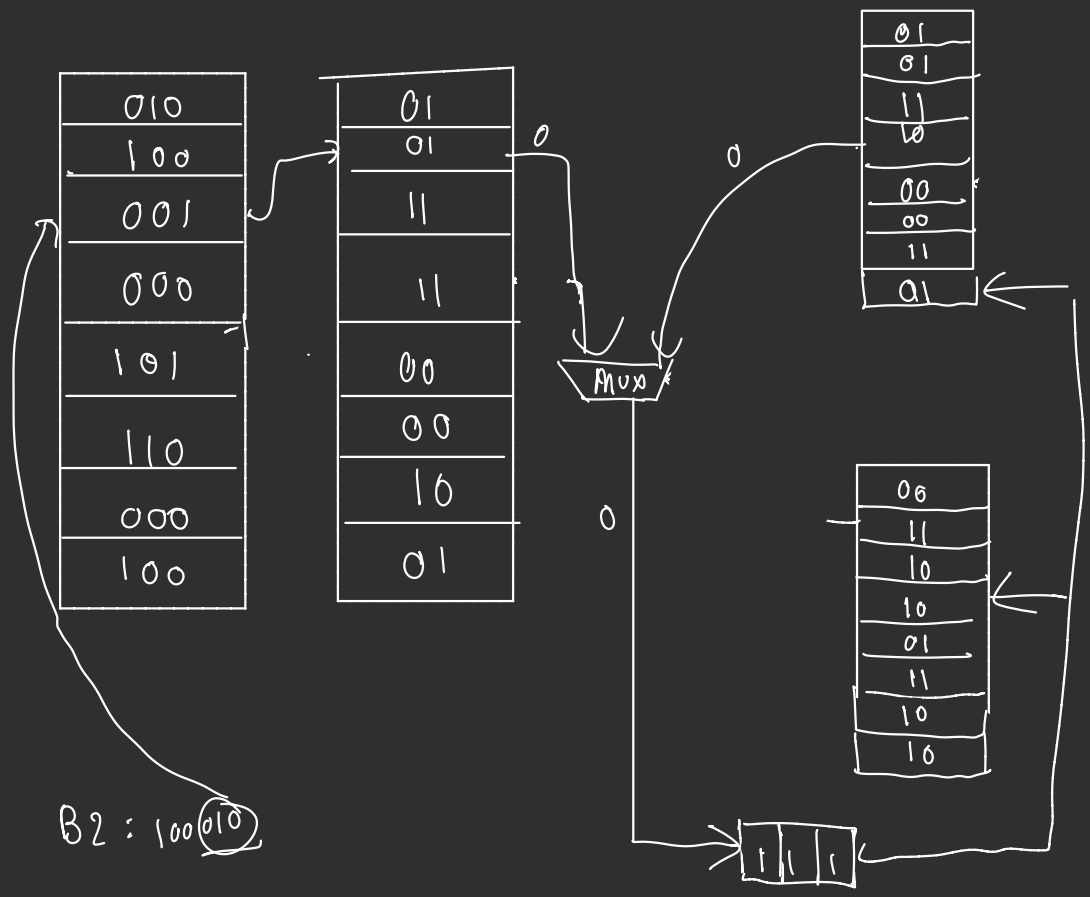
[Missed Class] ⇒ Correlating Branch Table

Made with Goodnotes

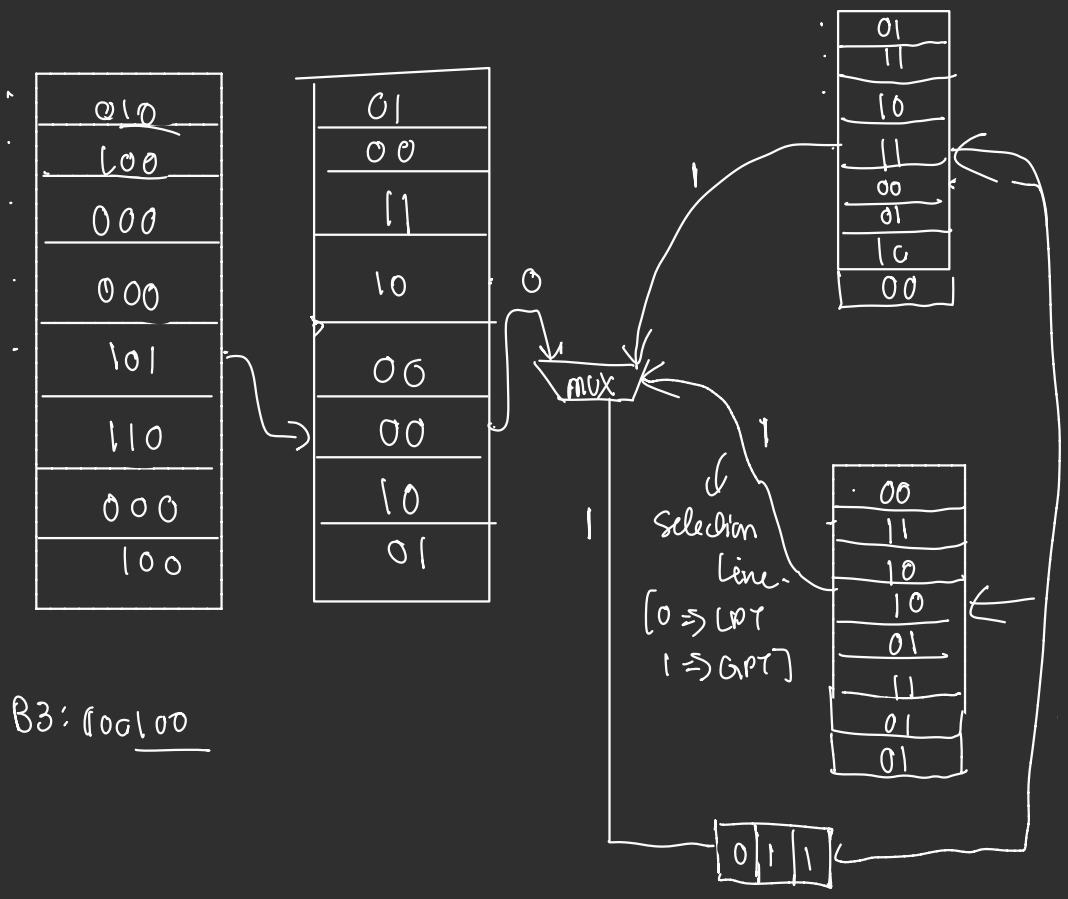
Tournament Branch Predictor



Made with Goodnotes

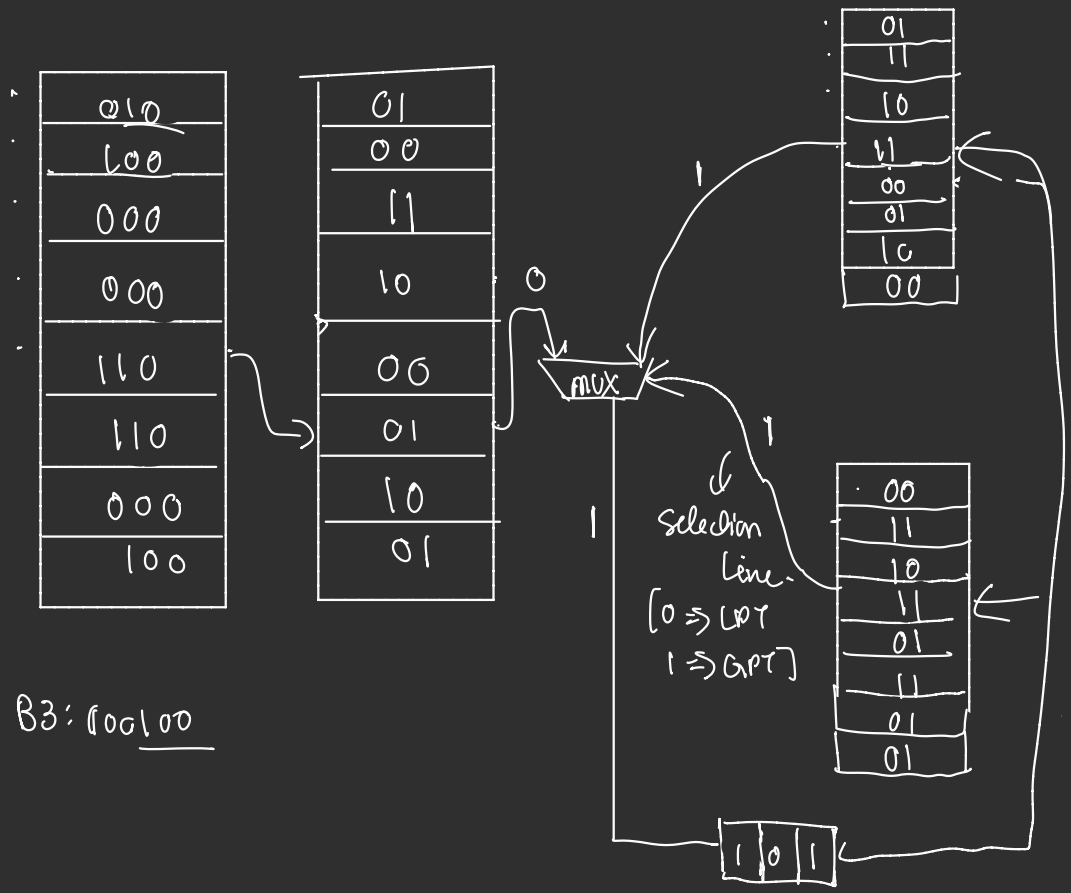


Made with Goodnotes



B3: 100100

Made with Goodnotes



B3: 100100

Made with Goodnotes

Static Register Renaming

I) Allocate Dest^{n} Register.

ARF [Architectural Register File]

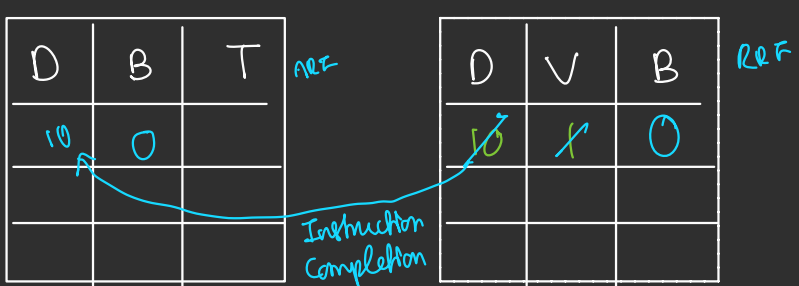
Data	Busy	Tag
	1	

RRF [Rename Register File] → Dependent on data.

Data	Valid	Busy → Depends on if register is free
	0	~~0~~ 1

1) Find Register in RRF with Busy = 0. [If Busy = 1, ignore]
2) Set RRF Busy = 1 & Valid = 0.
3) Map ARF Tag with RRF → EX starts now.
4) Set ARF Busy = 1.

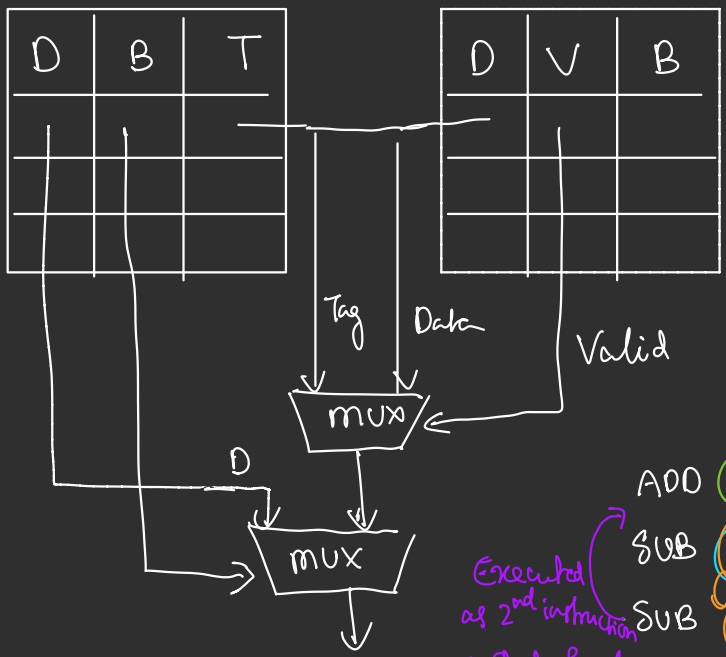
II) Update Register.

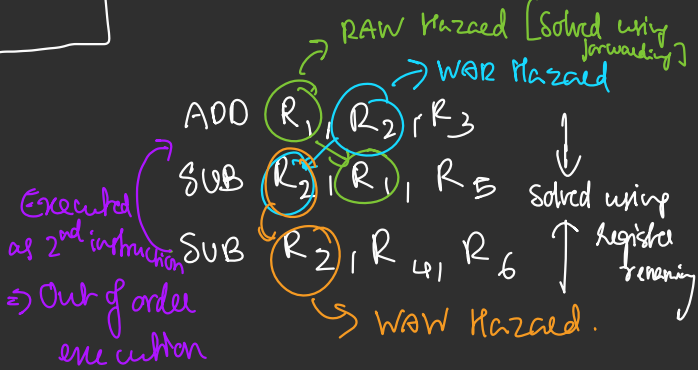


- Instruction finishes, 1) Update RRF Data field & Set V = 1!
- Instruction Completion:
  - Commit the Instruction:
    1) Write Data field to ARF
    2) Set ARF & RRF Busy = 0.

Made with Goodnotes

(iii) Source Read. → Decides from where to Read the data, ARF or RRF.





1) Completion:

If Busy = 0 the read data from ARF.

2) Finishes:

If Valid = 1, then Read data from RRF, else send tag info to reservation station

↳ Used for out of order execution.

Made with Goodnotes

ADD R_1, R_2, R_3
SUB R_3, R_2, R_1

MUL R_1, R_2, R_3
DIV R_2, R_1, R_3

RR: R_4, R_5, R_6, R_7

ADD R_4, R_2, R_3
SUB R_5, R_2, R_4
MUL R_6, R_2, R_5
DIV R_7, R_6, R_5

[missed 2 classes]

Made with Goodnotes

Instruction Level Parallelism

1) Compiler Based → Static
2) H/W based → dynamic

Compiler Based → Reorders instructions based on dependencies. The reordered instructions are given to the hardware.

H/W based → H/W takes care of reordering.

Parallelism → reduces CPI [$Pipeline_{CPI} = Ideal_{CPI} + Stall(S,D,C)$]

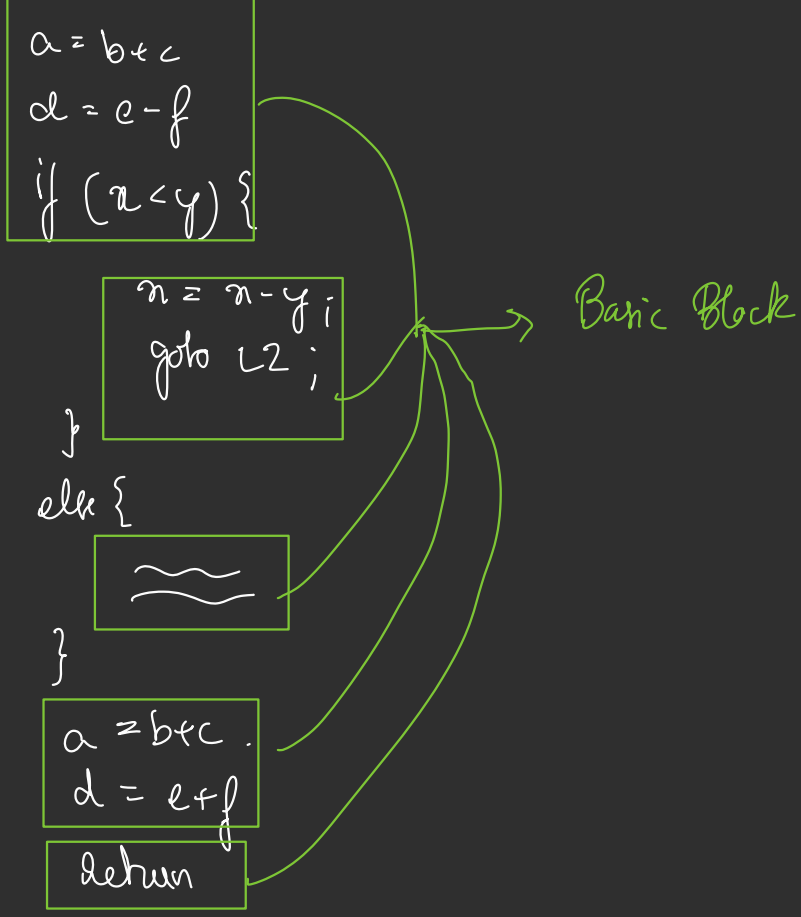
Basic Blocks

- Straight lines of code w/o any branches except the entry & no branches out except at the exit.

Eg:

```
a = b+c          → 1st basic block.
d = e-f
if (a<d) {
    y = x-y      → 2nd basic block
}
else {
    z = w-x;     → 3rd basic block
}
w = x+y;         → 4th basic block.
```

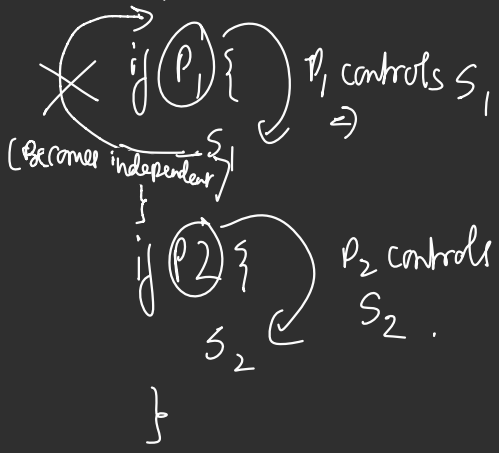
Made with GoodNotes



- Disadv: Contain limited no. of instructions ⇒ limited ILP (elism).
- WAR & WAW hazards can arise in instructions ⇒ order must be preserved.
  - WAR → Antidependence
  - WAW → output dependence
- RAW [data dependency] ⇒ stall.

Made with Goodnotes

Control Dependencies



① DADDU R_1, R_2, R_3

BEQZ R_2, L_1

LW R_1 0(R_2)

L_1: Add R_4, R_1, R_2.

② DADDU R_1, R_2, R_3

BEQZ R_4, Skip

DSUB R_1, R_5, R_6

SKIP: OR R_7, R_1, R_3

③ DADDU R_1, R_2, R_3

BEQZ R_4, Skip

DSUB R_4, R_5, R_6

PADD R_5, R_6, R_7.

Skip OR R_7, R_8, R_7

Control point.

Made with Goodnotes

Instruction producing result	Instruction using result	Latency.
IP ALU operation	FP ALU operation	3
FP ALU operation	Store double	2
Load Double	FP ALU O/p	1
11	Store double	0

```
for (int i = 999; i > 0; i = i - 1) {
    x[i] = x[i] + s;
}
```

→ Contains x[i]

```
Loop: LD F0, 0(R1)
      [1 stall]
      ADD F4, F0, R2      → Contains s.
      [2 stalls]
      SD F4, 0(R1)
      DADDUI R1, R1, #-8  → Decrement by 8 bytes.
      [1 stall]
      BNE R1, R2, Loop
```

4000 stalls ∈ { 4 stalls. 1000 times loop execute }

Made with Goodnotes

Reordering:

LD
DADDUI
ADD
Stall
Stall
SD
BNE

} → 1000 stalls to 2000 stalls.

Loop Unrolling

PLEASE TURN OVER

Made with Goodnotes

Loop: LD $F_0, 0(R_1)$ → 1 stall
ADDD F_2, F_0, R_2 → 2 stall
SD $F_2, 0(R_1)$
} 3 stalls

LD $F_4, -8(R_1)$
ADDD F_6, F_4, R_2
SD $F_6, -8(R_1)$
} 3 stalls

LD $F_8, -16(R_1)$
ADDD F_{10}, F_8, R_2
SD $F_{10}, -16(R_1)$
} 3 stalls

LD $F_{12}, -24(R_1)$
ADDD F_{14}, F_{12}, R_2
SD $F_{14}, -24(R_1)$
} 3 stalls

DADDUI $R_1, R_1, \#-32$
BNZ $R_1, R_2,$ LOOP → 1 stall.

1 iteration ⇒ 4 elements.

∴ 1000 iterations → $250 \times (3 \times 4 + 1)$
$= 250 \times 13$
$= 2575$

Made with Goodnotes

Loop Unrolling + Reordering

```
LD F0
LD F4
LD F8
LD F12
ADD1
ADD2
ADD3
ADD4
SD
SD
DADDUI
SD
SD
BNEZ
```

No stalls

Loop Unrolling & Reordering performed by compiler. After loop unrolling & reordering the code is fed to the h/w for inorder execution.

Disadvantages → Loop Unrolling ⇒ Large no. of instructions in cache, could lead to instruction

→ Loop Unrolling ⇒ Leads to large register usage

Made with Goodnotes

Dynamic Scheduling -> H/W

Rearrange the execution order of instructions.

Advs → Compiler doesn't have knowledge of microarchitecture
↳ Handles cases where dependencies are unknown at compile-time

Disadv → H/W complicates exceptions.

Limitation
→ Inorder Pipeline
→ Instruction issued in program order
→ If instruction is stalled, no later instruction can proceed.

DIV F_1, F_2, F_3
⇒
ADD F_4, F_1, F_5

MUL F_7, F_9, F_{10} → Although has no dependencies with the first 2 instructions, only executes after first 2 instructions are done in inorder pipeline

Issuing in Dynamic Schedule

1) Check for structural hazards ⇒ Allows for reordering.
2) Wait for absence of data hazard.
3) Use inorder instruction issue but wait for an instruction to

Made with Goodnotes

begin execution as soon as data operands are available

4) Out of order execution

5) Possibility of WAW & WAR hazard

6) Possibility for imprecise exception → Precise Exception
   - → Current instruction
   - Leads to exception, later instructions are stalled until exceptions are handled.
   - → Previous instructions are allowed to finish execution

precise ⇒ execution finished
imprecise ⇒ flushed

DIV
ADD
SUB
MUL

Exception ← MUL

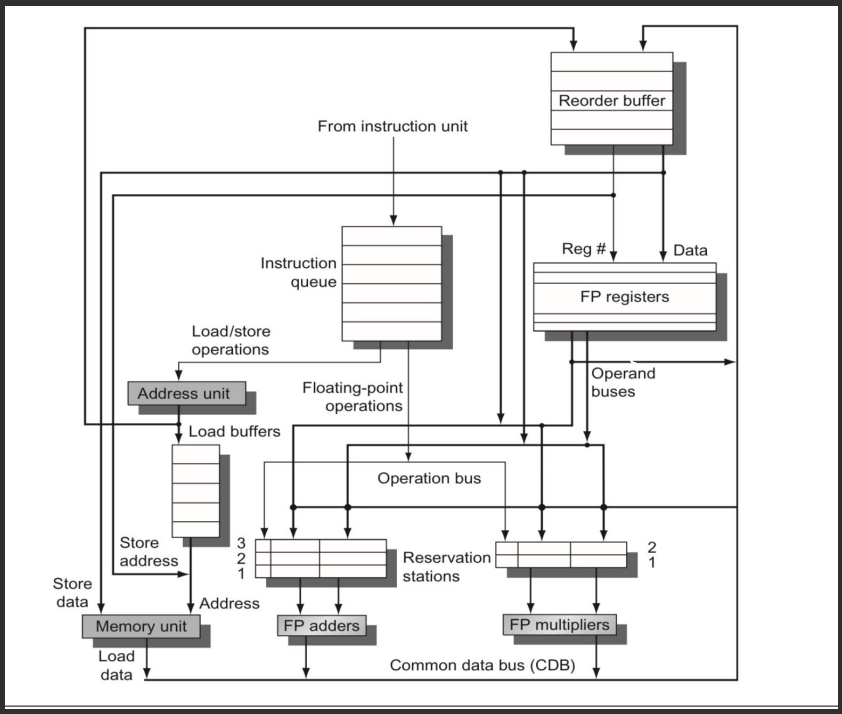
DIV D F0, F1, F2 → Divide by zero
ADD D F3, F0, F4
SUB D F8, F9, F10
MUL D F3, F14, F15

- precise ⇒ stalled
- imprecise ⇒ Complete before division.

Instruction Decode
- → Issue [in order]
- → Read operands [OOO]

Made with Goodnotes

Tomasulo's Algorithm & Done? Score boarding tech

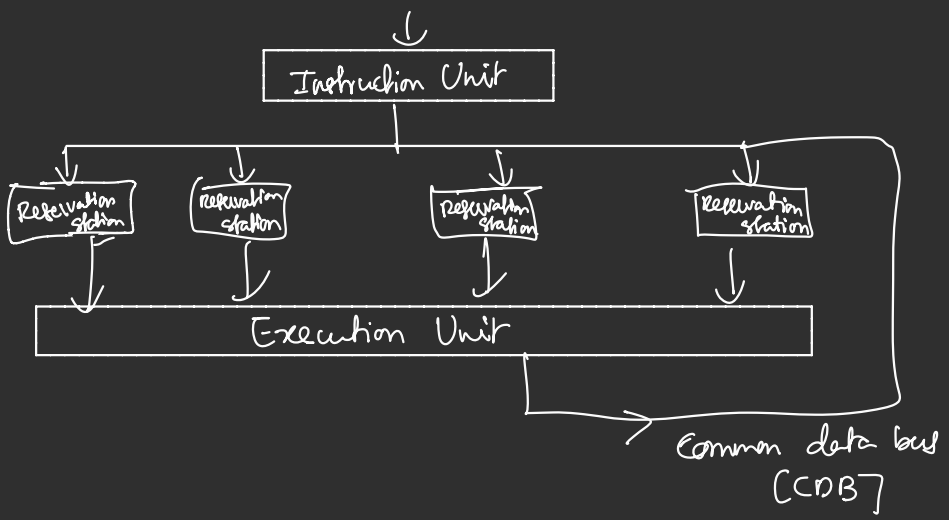


[missed Class]

Made with Goodnotes

Dynamic Scheduling Continuation

- Out of order execution & completion.



1) Fetch

2) Check Reservation Station [Free ⇒ store]

Execution

1) Operands are ready ⇒ execute, otherwise wait.
2) Load/Store user buffers
3) Tomasulo's Algorithm can only handle data & structural hazard, cannot deal with control hazard.

Write

CDB → loads data in both reservation station & floating point registers.

Made with Goodnotes

ADD R_1, R_2, R_3

ST $R_1, [R_4 + 50]$

ADD R_1, R_6, R_7

SUB R_8, R_5, R_9

ADD R_1, R_8, R_{10}

Register Status Indicator [RSI]

Registers:	1	2	3	4	5	6	7	8	9	10	11
Status:	~~0~~ 5	~~3~~ 5	0	0	0	0	0	0 ~~0~~ 4	0	0	0

Execution Unit

Unit No:	1	2	3	4	5	6
	ADD_1, E	$ST, W1$	ADD_2, E	SUB, E	$ADD_3, W4$	

Reservation Station Fields

1) Operation [opr]

2) V_j } => Register file

3) V_k => Load stores value in V_k

4) Q_j } if $Q_j = Q_k = 0$ => data is read from register file

5) Q_k non zero => reading from execution unit.

6) A -> Address for Load & store [Initially contains immediate address, then stores effective address]

7) Busy

Made with Goodnotes

LD R_1, [R_4 + 50]

LD R_2, [R_3 + 16]

MUL R_{11}, R_8, R_6

SUB R_7, R_{11}, R_8

DIV R_{11}, R_9, R_{10}

ADD R_1, R_{11}, R_{12}

RS	1	2	3	4	5	6	7	8	9	10	11
Unit name	~~LD1~~	LD2	0	0	0	0	S1	0	0	0	0
	~~M1~~										
	~~D1~~										
	A1										

Register file

Execution unit

Name of the unit	Oper	Busy	V_j	V_k	Q_j	Q_k	A
LD1	Load	Yes	—	—	—	—	[R_4 + 50]
LD2	Load	Yes	—	—	—	—	[R_3 + 16]
M1	MUL	Yes	Reg R_5	Reg R_6	0	0	—
S1	SUB	Yes	0	Reg R_8	M1	0	—
D1	DIV	Yes	Reg R_9	Reg R_{10}	0	0	—
A1	ADD	Yes	Reg R_{11}	Reg R_{12}	0	0	—

Made with Goodnotes

LD F6, 32(R2)

LD F2, 44(R3)

MUL F0, F2, F4

SUB F6, F2, F6

DIV F10, F0, F6

ADD F6, F8, F2

RS1	F0	F2	F4	F6	F8	F10	R2	R3
	~~0~~ MUL1	~~0~~ LD2	0	~~0~~ ~~LD1~~ ADD2	~~0~~ ADD1	0	0	0

Name of unit	opr	Busy	V_j	V_k	Q_j	Q_k	A
LD1	Load	Yes	—	—	—	—	32(R2)
LD2	Load	Yes	—	—	—	—	44(R3)
ADD1	SUB	Yes	0	0	LD2	LD1	—
ADD2	ADD	Yes	0	0	LD2	ADD1	—
ADD3	—	No	—	—	—	—	—
MUL1	MUL	Yes	0	F4	LD2	0	—
MUL 2	DIV	Yes	0	0	MUL1	LD1	—

Made with Goodnotes

Q:

LD F6, 32(R2)

LD F2, 44(R3)

MUL F0, F2, F4

SUB F6, F2, F6

DIV F10, F0, F6

ADD F6, F8, F2

LD/ST → 2 cycles

ADD/SUB → " "

MUL → 10 "

DIV → 40 "

Execution

Issue	Complete	Write Result
1	3	4
2	4	5
3	15	16
4	7	8
5	56 10	57 11
6		

Cycle 1

RS1	F0	F2	F4	F6	F8	F10	R2	R3
	∅	∅	0	∅	∅	0	0	0
	MUL1	LD2		~~LD1~~ ADD2	ADD1			

Name of unit	opr	Busy	V_j	V_K	Q_j	Q_K	A
LD1	Load	Yes	—	—	—	—	32(R2)
LD2	Load	Yes	—	—	—	—	44(R3)
ADD1	SUB	Yes	0	0	LD2	LD1	—
ADD2	ADD	Yes	0	0	LD2	ADD1	—
ADD3	—	No	—	—	—	—	—
MUL1	MUL	Yes	0	F4	LD2	0	—
MUL 2	DIV	Yes	0	0	MUL1	LD1	—

Made with Goodnotes

ADD R_1, R_2, R_3
ST $R_1, [R_4 + 60]$
ADD R_1, R_6, R_7
SUB R_8, R_5, R_9
ADD R_1, R_8, R_{10}

LD/ST → 2 cycles
ADD/SUB → " "
MUL ⟶ 10 "
DIV ⟶ 40 "

RSI:

R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
0	0	0	0	0	0	0	0	0	0

Name of unit	opr	Busy	V_j	V_k	Q_j	Q_k	A
ADD 1							

[missed 2 classes]

Made with Goodnotes

Superscalar Processors

1) Very Long Instruction Word : Multiple operations bundled together [VLIW]

- Int | FP | Mem | FP → Fixed format.
- → Adv → Increases ILPism.
- → Disadv → Some slots may not be filled as some instructions maybe dependent.
  - → Branch hazards dealt with unrolling ⇒ code size ↑ & memory ↑.
- → Requires handling of:
  - → Dependencies b/w instructions
  - → Multiple reads
  - → Complex circuits to figure out dependencies
  - → Multiple commits
  - → Wider COB
- → Usually contains 1024 bits

Made with Goodnotes

Q:

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R1, R1, #-8
Stall
BNE R1, R2, LOOP
```

Assume:

- One Int
- 2 FP
- 2 Mem reference
- 7 times

	Mem ref 1	Mem ref 2	FP 1	FP 2	INT 1
1	LD F0, 0(R1)	LD F6, -8(R1)			
2	LD F10, -16(R1)	LD F14, -24(R1)			
3	LD F18, -32(R1)	LD F22, -40(R1)	ADD F4, F0, F2	ADD F8, F6, F2	
4	LD F26, -48(R1)	—	ADD F_{12}, F_{10}, F_2	ADD F_{16}, F_{14}, F_2	
5	—	—	ADD F_{20}, F_{18}, F_2	ADD F_{24}, F_{22}, F_2	
6	SD F4, 0(R1)	SD F8, -8(R1)	ADD F_{28}, F_{26}, F_2	—	⑦DADDUI R1, R1, #56
7	⑪ F_{12}, -16(R1)	SD F16, -24(R1)			
8	F_{20}, -24(R1)	SD F24 -			
9	F_{28}				BNE R1, R2, LOOP

Made with Goodnotes

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R3, R1, #-8
Stall.
```

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R5, R1, #-8
Stall.
```

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R7, R1, #-8
Stall.
```

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R9, R1, #-8
Stall.
```

Made with Goodnotes

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R11, R11, #-8
Stall.
```

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R13, R11, #-8
Stall.
```

```
LD F0, 0(R1)
Stall
ADDD F4, F0, F2
Stall
Stall
SD F4, 0(R1)
DADDUI R15, R11, #-8
Stall.
BNE R15, R2, LOOP
```

Made with Goodnotes

Iteration no.	Instⁿ	Issues at CC Number	Execution @ CC No.	mem access @ CC no	Write CDB @ CC no.	Comments	Comm it
		Spec	Spec		Spec		
1	LD $R_2, 0(R_1)$	1 2	2	3	4 4	Wait for LW	5
	DADDIU $R_2, R_2, \#1$	1 5	5		6 6	" DADD	7
	SD $R_2, 0(R_1)$	2 3	3	7		Execute directly	7
	DADDIU $R_1, R_1, \#8$	2 3	3		4 4	Wait for DADD	8
	BNE R_2, R_3, LOOP	3 7	7			" BNE	8
						" LW	
2	LD $R_2, 0(R_1)$	4 5	8	9	10 7		9
	DADDIU $R_2, R_2, \#1$	4 8	11		12 9		10
	SD $R_2, 0(R_1)$	5 6	9	13			10
	DADDIU $R_1, R_1, \#8$	5 6	8		9 7		11
	BNE R_2, R_3, LOOP	6 10	13				11
3	LD $R_2, 0(R_1)$	7 8	14	15	16 10		12
	DADDIU $R_2, R_2, \#1$	7 11	17		18 12		13
	SD $R_2, 0(R_1)$	8 9	15	19			13
	DADDIU $R_1, R_1, \#8$	8 9	14		15 10		14
	BNE R_2, R_3, LOOP	9 13	19				14

W/o Speculation & with Speculation

Made with Goodnotes

With Speculation

Issue	Execute	Read Access	Write CDB	Commits	Comment.

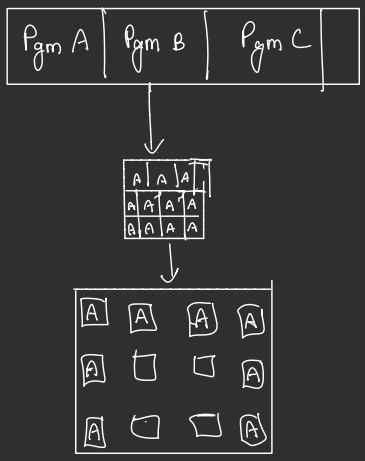
Made with Goodnotes

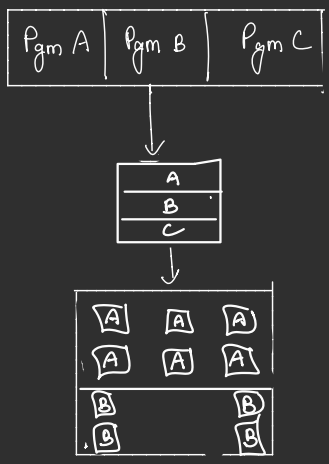
LD $R_2, \sigma(R_1)$

Perf tool?
Assignment?

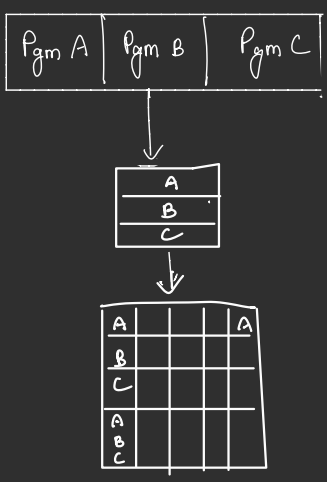
Made with Goodnotes

Multithreading Architecture For Superscalar Processors

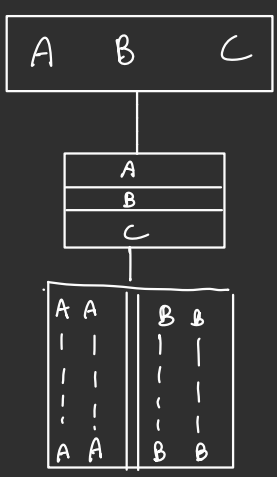




Coarse-Grain multithreading

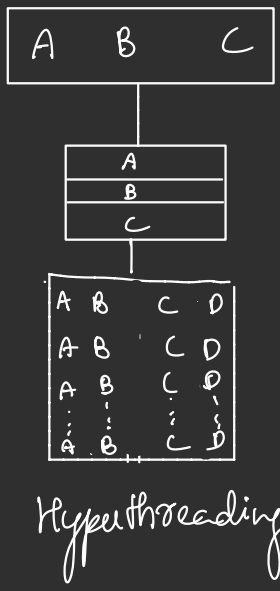


Fine grain multithreading



Multiprocessing

Made with Goodnotes



Hyperthreading

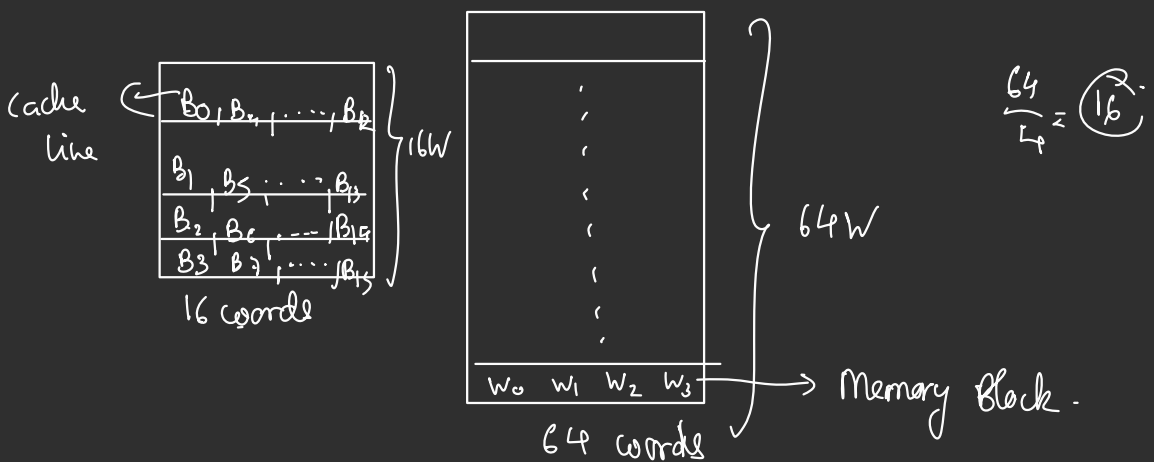
[missed class]

Made with Goodnotes

Register → Lowest part of hierarchy

↓

Magnetic Tape → Highest part of hierarchy.



$\frac{64}{4} = 16$

. Line Size = Block size

no. of lines = $\frac{\text{Cache size}}{\text{Block size}} = \frac{4 \times 16W}{4W} = 4$ lines

Direct Mapping → k Mod m

↓ Block No. ↓ no. of lines.

Made with Goodnotes

Physical Address.

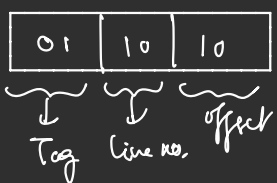
↳

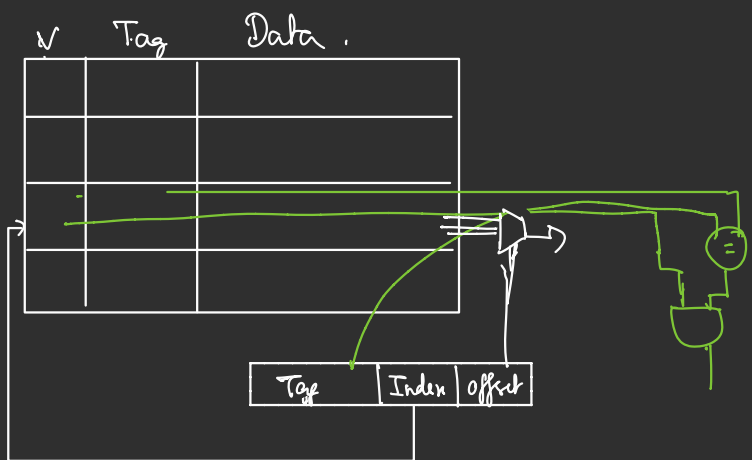
Tag ⇒ Identifies block, 16 blocks in memory.

↳ Each block has 4 words ⇒ required 2 bits.

011010 ⇒ 6th block, 2nd word.

26+16 = 26th word.





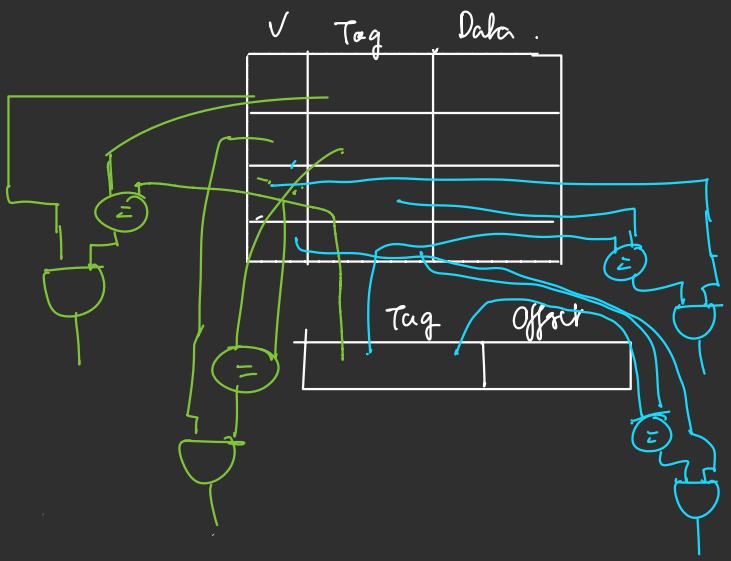
Direct Mapping

↳ Disadv → Conflict Misses

Made with Goodnotes

Fully Associative

- Each line can have any block.



Advs:
- Increases no. of hits
- Decreases conflict misses

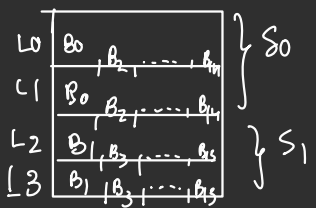
Disadv:
- Design complexity

Set Associative Mapping

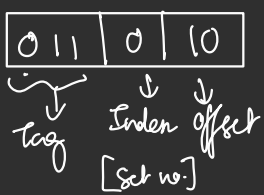
- Placement of Blocks → K Mod n
  - K → Block no.
  - n → Set no.

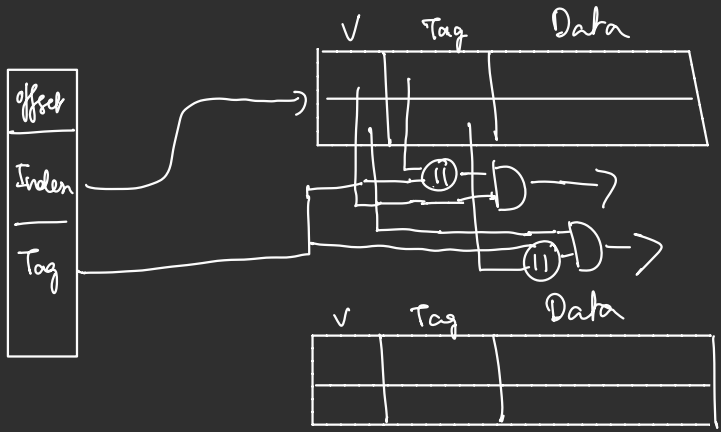
- K-way set associative ⇒ no. of sets = $\frac{\text{no. of lines}}{K}$

Made with Goodnotes



2-way set associativity





Which Cache placement policy leads to internal & external fragmentation.

Cache Replacement Policy

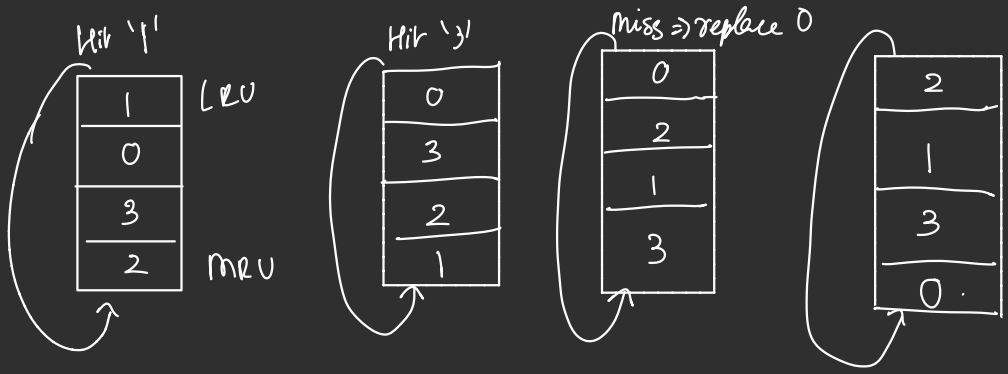
- Random, FIFO, LIFO, LRU, Optimal, Pseudo LRU, MRU etc.

LRU & MRU → Uses a bit to identify which line is LRU & MRU. [For 2 way]

[LRU ⇒ 0
MRU ⇒ 1.

Made with Goodnotes

For more than 2 way set associativity, use counters.



Optimal → look at future cache accesses for replacement.

Q: Consider a direct mapped cache. Cache size = 32 KB
Block size = 32 Bytes.
Address size = 32 bit.

Find no. of bits for cache index?
Tag bits?

$$\frac{32 \times 2^3}{32} =$$

3 bits for cache index
5 bits for offset.
24 bits for tag.

Made with Goodnotes

Byte Addressable MM = 256 MB

Cache Size = 1 MB

Block Size = 128 B.

2 way S-A

Physical Address?

Tag ?

No. of comparators?

4 MB = MM

64 B = Block size.

10 bits → Tag.

4 way S-A

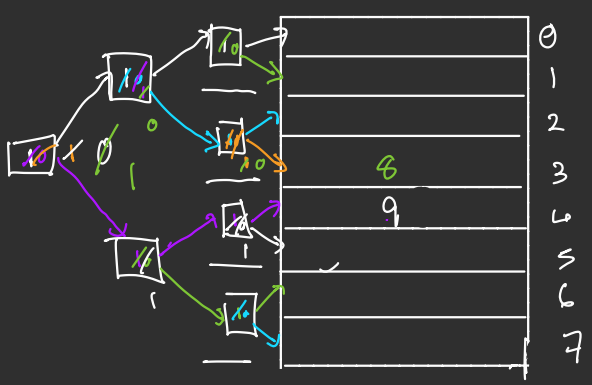
Physical Address Split?

Cache Size ?

Pseudo LRU

L→ Binary Tree.

0, 1, 2, 3, 4, 5, 6, 7, 2, 1, 9, 8.



0 ⇒ down

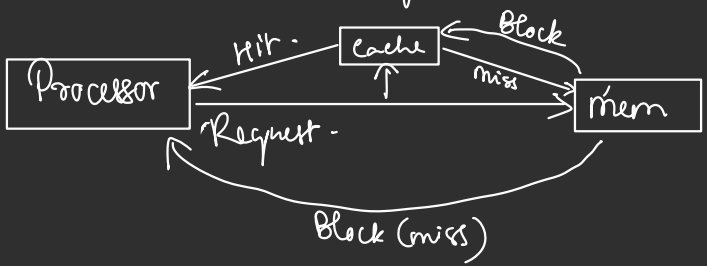
1 ⇒ up.

Made with Goodnotes

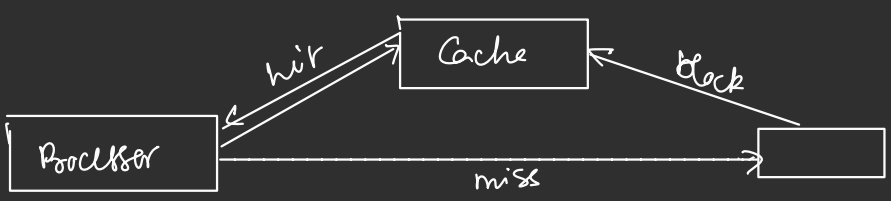
NRU [Not Recently Used]

↳ Uses a bit [0|1] to indicate 0/1.

Look Aside & Look Through



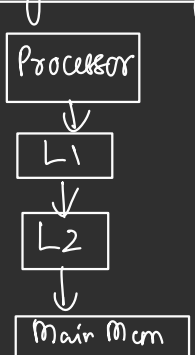
during cache hit, main memory also searches for block ⇒ waste of time ⇒ Look Through.



Look Aside.

Different Organization of Cache

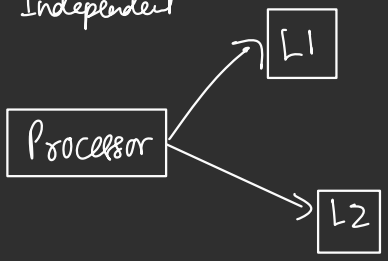
1) Hierarchical



↳ Hit Time: $H_1 T_1 + (1-H_1) H_2 (T_1 + T_2)$

Made with Goodnotes

II) Independent

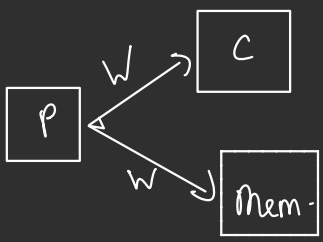


$H_1 T_1 + (1-H_1) H_2 T_2$.

↳ Faster but more complex.

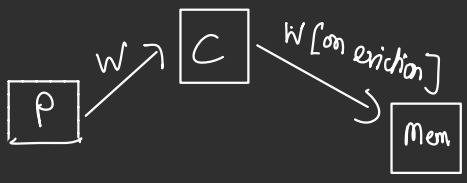
Write Policies

Write Hit
- Write Through
- Write Back



Write Through

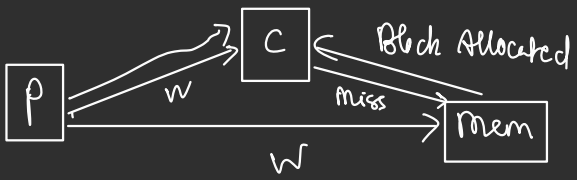
- Disadv
  - ↳ During loops always writing into memory



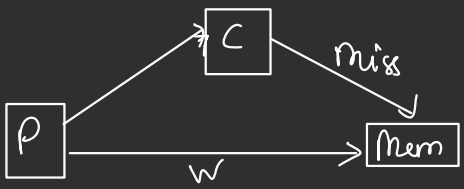
Write Back.

Write Miss
- Write Allocate
- No Allocate.

Made with Goodnotes

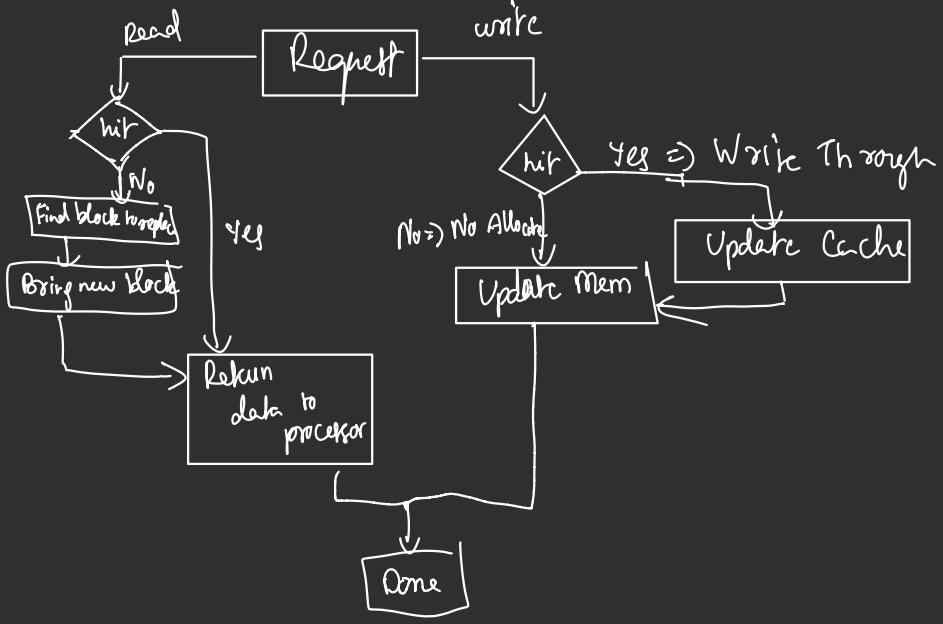


Write Allocate.

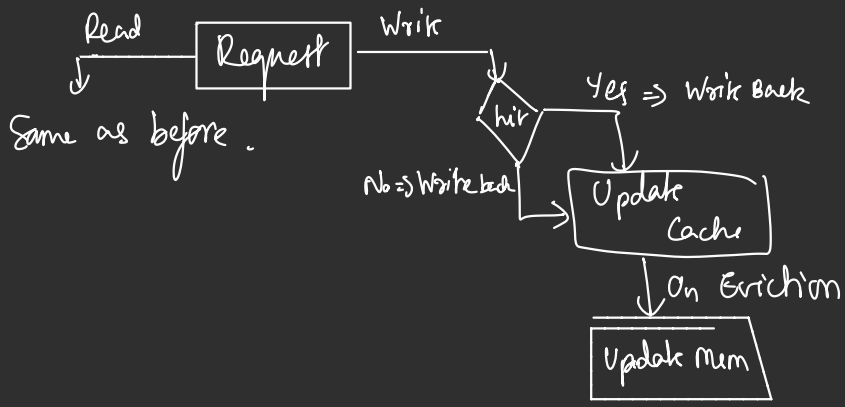


No allocate.

Write Through + No allocate } Combinations used,
Write Back + Write Allocate }



Made with Goodnotes



Cache Optimization

- Average Memory Access Time [AMAT] = HT + MR × MP. (HT: Hit Time; MP: Miss penalty; MR: Miss Rate)

- To increase performance ⇒ Decrease HT & decrease MR & MP.

- Ways to increase Cache performance

1) Larger Block Size ⇒ reduces compulsory miss.
   - Utilizes spatial locality.
   - Disadv:
     - → Increase in conflict misses [reduces set associativity]
     - → Increases hit time.
     - → Reduces miss rate

2) Larger Cache
   - → more no. of hits
   - → Reduces capacity misses.
   - → Disadv → Cache covers more area, increases cost [uses SRAM], more power.

Reduces miss rate

Made with Goodnotes

3) Higher Associativity

- Increase Hits.
- Decrease conflict misses.
- Disadv:
  ↳ Hit time increases. ⇒ don't use full associativity.
- Reduces miss rate.

4) Multi-Level Cache Memory.

- Decreases miss penalty
- L1 → small size, matches speed of processor.
  L2 → larger size, reduces access to main memory / reduces time to transfer data b/w main memory & processor.
- 2 different organizations:
  i) Inclusive → if cache size = 64W ⇒ L1 & L2 share the 64W.
  ii) Exclusive → if cache size = 64W ⇒ L2 = 64W, L1 = 8W. (They don't share the size].
- Local miss rate → miss rate for one level of cache.
- Global miss rate → miss rate for the entire cache system.
- Reduces miss penalty time

Made with Goodnotes

5) Prioritize Read Miss over write.

- Whenever there is a read miss, evict the dirty block and then write the required block into the cache line.
  - ↳ Increases CPU waiting time. [Disadv]
- Fixed by writing the dirty block into a buffer. [Reduces miss penalty time].

Advanced Optimizations

1) Way Prediction
   - ↳ Implemented in set associative mapping
   - ↳ Predicts the cache line to access.
   - ↳ Each line has an extra bit called predictor bit.

2) Victim Cache.
   - ↳ Recently evicted blocks are loaded into 'victim cache'.
   - ↳ If next access is the block which was recently evicted, then we will find it in the victim cache & re-load it into the cache.

3) Pipeline Cache
   - ↳ Pipeline the stages of accessing the cache.
   - ↳ Steps: Indexing, Tag comparison, Hit/misses, data fetching.

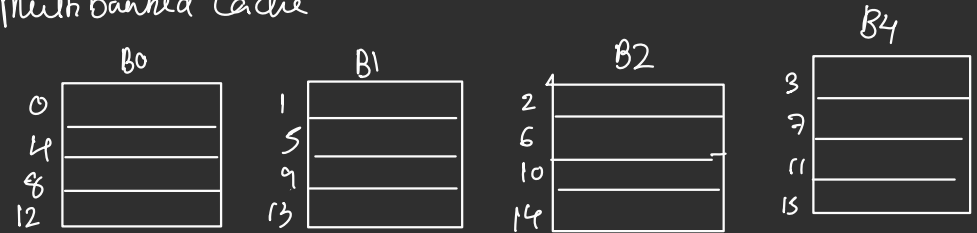
Made with Goodnotes

Indexing	Tag Comparison	H/M	Data Fetch
L0			
L1	L0		
L2	L1	L0	

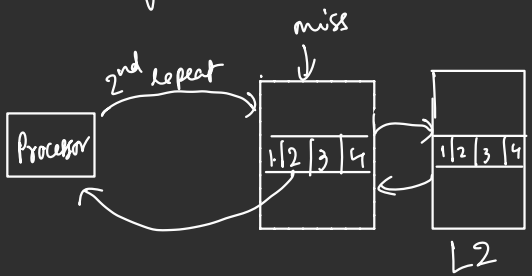
4) Non-Blocking Cache

- Cache handles all requests without any waiting.
- Miss Status Holding Register.

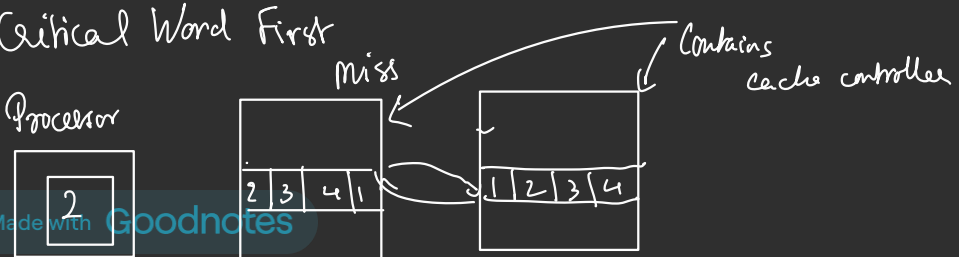
5) Multibanked Cache



6) Early Restart



7) Critical Word First



Made with Goodnotes

8) Hardware Prefetching

→ Deals with instructions.

→ When fetching one instruction, prefetches the next instruction as well.
- one instruction → stored in cache.
- next instruction → stored in stream buffer.

→ When next instruction requested, the instruction is loaded from the stream buffer to the cache.

CT - 2

- Exceptions
- Multicycle (FP)
- Static Scheduling
  - Basic Blocks
  - Loop Unrolling
    - → Strip mining.
  - Scheduling
- Dynamic Scheduling
  - Tomasulo's Algorithm
  - Speculation
  - Superscalar [VLIW]
- Cache Memory
  - Mapping techniques
  - Basic optimization
  - Advanced "

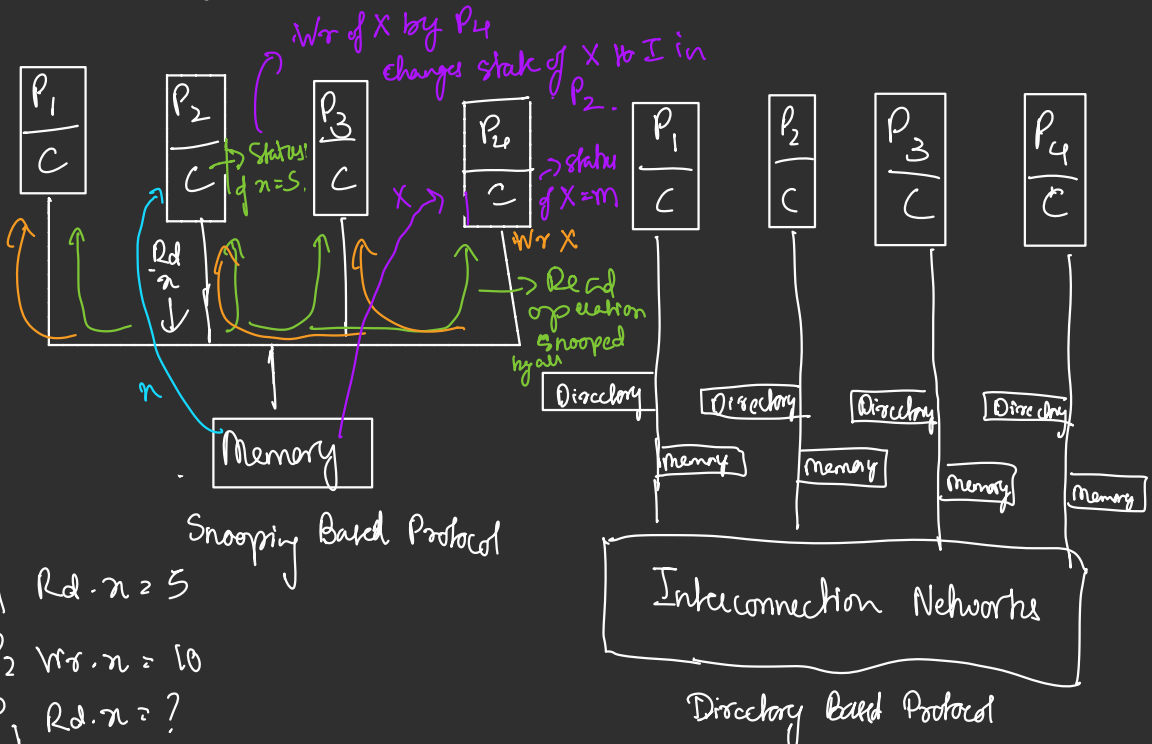
Made with Goodnotes

- Write Strategy
  - -> Flow chart
- Locality
- Block (re) placement
- Design Issues

Cache Coherency Protocol

-> Modified Shared Invalid

1) Snooping protocol -> Shared memory [MSI, MESI]
2) Directory Based Coherency -> Distributed memory.



P_1 Rd. x = 5

P_2 Wr. x = 10

P_1 Rd. x = ?

Made with Goodnotes

SNOOPING BASED PROTOCOL

If there is a read after write in MSI [Writeback], it updates the value in memory as well as the processor that modified the value sends the value to the processor requesting it. The state of X in the modifying processor becomes S now.

If the values held by the caches are S, then the value is taken from the main memory.

If same processor first writes, then reads, there is no change in its state? Only if another processor reads, then it becomes S, if another processor writes, state becomes I.

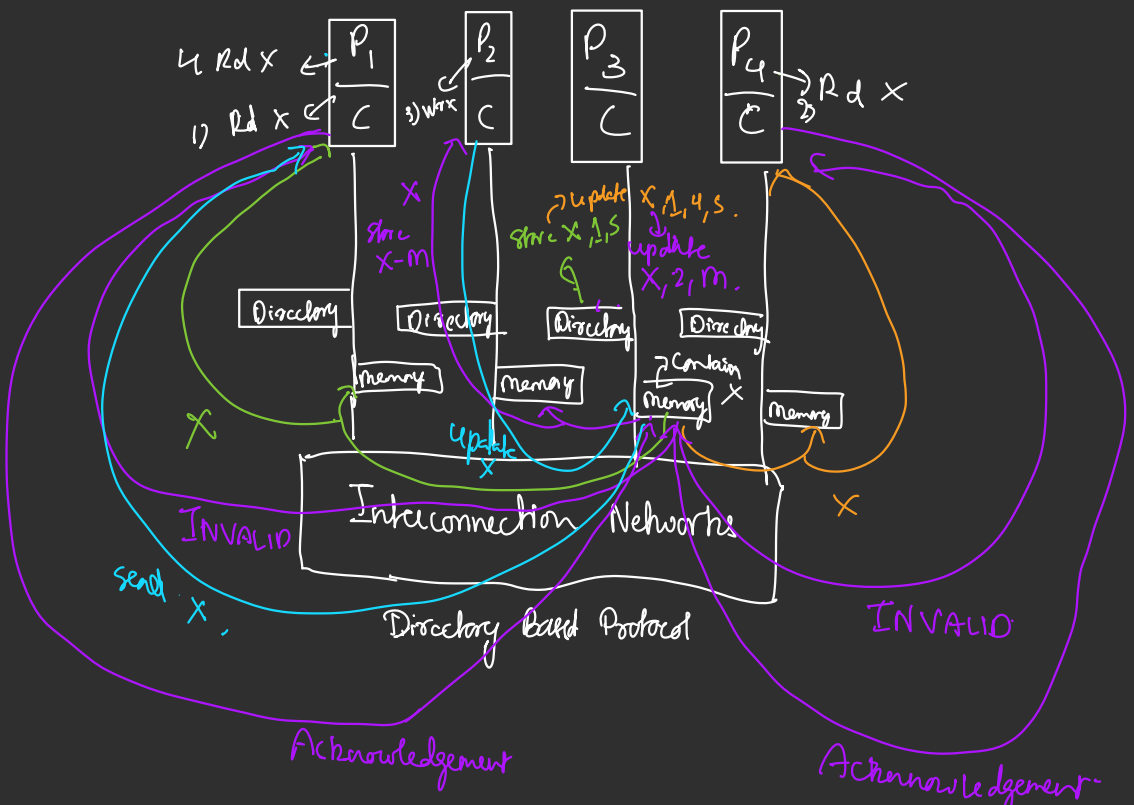
Q:

Request	Cache Mem	Request on the bus.	Who Responds	State in C_1	C_2	C_3	C_4
P_1 Rd X	miss	Rd X	Mem	S	I	I	I
P_2 Rd X	Miss	Rd X	Mem	S	S	I	I
P_2 Wr X	Miss	Wr X	—	I	M	I	I
P_3 Wr X	Miss	Wr X	P2	I	I	M	I
P_3 Rd X	Hit	—	P_3	I	I	M	I
P_4 Rd X	Miss	Rd X	P_3	I	I	S	S

Made with Goodnotes

If a processor first reads, then write, it is still a miss.

Directory Based Protocol



- more latency.
- ACKs can be lost ⇒ will not allow for value to be modified.

Made with Goodnotes

Q:

Request	Cache Mem	Message	Dir State	State in C_1	C_2	C_3	C_4
P_1 Rd X	miss	Rd Req to Dir	X, S: 1	S	I	I	I
P_2 Rd X	miss	"	X, S: 1,2	S	S	I	I
P_2 Wr X	miss	Upd Req to Dir, Send Inv to P_1	X, M: 2	I	M	I	I
P_3 Wr X	miss	Wr req to Dir	X, M: 3	I	I	M	I
P_3 Rd X	Hit	—	X, M: 3	I	I	M	I
P_4 Rd X	miss		X, S: 3,4	I	I	S	S

Same states as snooping

Rd → Rd ⇒ Mem

Wr → Rd ⇒ Processor

$Wr_{P_i} \rightarrow Rd_{P_i}$ ⇒ No change in state

$Wr_{P_i} \rightarrow Rd_{P_j}$ ⇒ State → S, update M.M.

Made with Goodnotes